

1000+ SOFTWARE ENGINEERING INTERVIEW QUESTIONS

Complete Preparation Guide with Answers, Code & Real-World Scenarios

2026 Edition - Freshers to Software Architects

Level: Intern / Fresher to Senior / Tech Lead / Architect

Targets: FAANG, product companies, high-growth startups, service companies

Style: Every question answered from first principles, with runnable code and expected output

HOW TO USE THIS BOOK

1. Read the question only. Answer out loud for two minutes. Time yourself.
2. Then read the answer. Score yourself: knew it / partial / new.
3. Type the code by hand. Do not copy-paste. Run it. Break it on purpose.
4. Answer the follow-ups without looking. These are what the interviewer actually asks next.
5. Night before the interview: read only the Revision Notes at the end of each section.

Difficulty calibration

Difficulty	Expected from	What failure signals

Easy	Intern / fresher	Red flag; usually ends the round
Medium	1-4 years experience	Down-level from mid to junior
Hard	5+ years / senior+	Not a reject alone; judged on reasoning quality

Every question uses this structure

Question and difficulty, Category, Answer, Example, Code Example, Output, Why Interviewers Ask This, Common Mistakes, Best Practices, Follow-up Questions,

Real-world Scenario.

Conventions

- SQL dialect is PostgreSQL 16+ unless the question names another engine.
- JavaScript is ES2024, Node 22 LTS, React 19, TypeScript 5.6+.
- Java is 21 LTS, Spring Boot 3.x. C# is 12 / .NET 8+.
- No proprietary or leaked interview content is reproduced. All questions are written from scratch against standard computer science material and publicly discussed themes.

TABLE OF CONTENTS

Section	Topic	Questions
1	SQL	150
2	MERN Stack (MongoDB, Express, React, Node)	120
3	Internship and Fresher Interviews	100
4	Computer Networks	100
5	System Design	120
6	HTML	60
7	CSS	100
8	JavaScript	150
9	TypeScript	80
10	C# and .NET	100
11	Java	120
12	Linux	100
13	Data Structures and Algorithms	150
14	Object-Oriented Programming	60
15	DBMS	80
16	Operating Systems	80
17	Software Engineering Practice	50
18	Behavioral and HR	100

SECTION 1: SQL

The Sample Database

Every SQL answer in this section runs against this schema. Load it once and every query in the section executes verbatim. Domain: a small e-commerce company.

```
+-----+      +-----+      +-----+
| customers |1-----*| orders  |1-----*| order_items |
+-----+      +-----+      +-----+
                        |1          *|
                        *          1|
                        +-----+      +-----+
                        | payments |    | products  |
                        +-----+      +-----+
                                      *|
                                      1|
                                      +-----+
                                      | categories | (self-referencing tree)
                                      +-----+

+-----+
| employees | manager_id -> employees.employee_id (self-referencing)
+-----+
```

```
-- =====
-- interview_db : sample schema for the SQL section
-- =====
DROP TABLE IF EXISTS payments, order_items, orders, products,
                        categories, customers, employees CASCADE;

CREATE TABLE customers (
  customer_id  SERIAL PRIMARY KEY,
  full_name    VARCHAR(100) NOT NULL,
  email        VARCHAR(150) NOT NULL UNIQUE,
  city         VARCHAR(60),
  country      VARCHAR(60) NOT NULL DEFAULT 'India',
  signup_date  DATE         NOT NULL DEFAULT CURRENT_DATE,
  last_support_contact TIMESTAMPTZ -- NULL = never contacted support
);

CREATE TABLE categories (
  category_id  SERIAL PRIMARY KEY,
  name         VARCHAR(60) NOT NULL UNIQUE,
  parent_id    INT REFERENCES categories(category_id)
);

CREATE TABLE products (
  product_id   SERIAL PRIMARY KEY,
```

```

    name          VARCHAR(120) NOT NULL,
    category_id    INT NOT NULL REFERENCES categories(category_id),
    unit_price     NUMERIC(10,2) NOT NULL CHECK (unit_price >= 0),
    stock_qty      INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
    discontinued   BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE orders (
    order_id       SERIAL PRIMARY KEY,
    customer_id    INT NOT NULL REFERENCES customers(customer_id),
    order_date     DATE NOT NULL,
    status         VARCHAR(20) NOT NULL
                CHECK (status IN ('PENDING', 'SHIPPED', 'DELIVERED', 'CANCELLED')),
    shipped_date    DATE,                -- NULL until it ships
    discount_pct   NUMERIC(5,2)         -- often NULL
);

CREATE TABLE order_items (
    order_id       INT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id     INT NOT NULL REFERENCES products(product_id),
    quantity       INT NOT NULL CHECK (quantity > 0),
    unit_price     NUMERIC(10,2) NOT NULL,    -- price snapshot at purchase time
    PRIMARY KEY (order_id, product_id)       -- composite primary key
);

CREATE TABLE payments (
    payment_id     SERIAL PRIMARY KEY,
    order_id       INT NOT NULL REFERENCES orders(order_id),
    amount         NUMERIC(12,2) NOT NULL,
    method         VARCHAR(20) NOT NULL,      -- UPI / CARD / NETBANKING / COD
    paid_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    balance        NUMERIC(12,2) NOT NULL DEFAULT 0
);

CREATE TABLE employees (
    employee_id    SERIAL PRIMARY KEY,
    full_name      VARCHAR(100) NOT NULL,
    department     VARCHAR(50) NOT NULL,
    salary         NUMERIC(10,2) NOT NULL,
    manager_id     INT REFERENCES employees(employee_id),
    hire_date      DATE NOT NULL
);

-- ----- seed data -----
INSERT INTO customers (full_name, email, city, country, signup_date, last_support_contact) VALUES
('Aarav Sharma', 'aarav@example.com', 'Bengaluru', 'India', '2024-01-15', '2025-03-02
10:12:00+05:30'),
('Diya Patel', 'diya@example.com', 'Mumbai', 'India', '2024-02-20', NULL),
('Rohan Verma', 'rohan@example.com', 'Delhi', 'India', '2024-03-05', NULL),
('Meera Iyer', 'meera@example.com', 'Chennai', 'India', '2024-06-11', '2025-01-19
09:00:00+05:30'),
('John Carter', 'john@example.com', 'Austin', 'USA', '2024-07-30', NULL),
('Sara Khan', 'sara@example.com', NULL, 'UAE', '2025-02-14', NULL);

INSERT INTO categories (name, parent_id) VALUES
('Electronics', NULL), -- 1
('Computers', 1), -- 2
('Laptops', 2), -- 3
('Accessories', 2), -- 4
('Home', NULL), -- 5
('Kitchen', 5); -- 6

INSERT INTO products (name, category_id, unit_price, stock_qty, discontinued) VALUES
('UltraBook 14', 3, 89999.00, 12, FALSE),
('ProBook 15', 3, 74999.00, 0, FALSE),
('Mechanical Keyboard', 4, 4999.00, 40, FALSE),
('USB-C Hub', 4, 2499.00, 75, FALSE),
('Wireless Mouse', 4, 1299.00, 5, TRUE),

```

```

('Air Fryer 4L',          6, 6999.00, 20, FALSE),
('Blender Pro',          6, 3499.00, 18, FALSE);

INSERT INTO orders (customer_id, order_date, status, shipped_date, discount_pct) VALUES
(1, '2025-01-10', 'DELIVERED', '2025-01-12', 10.00),
(1, '2025-02-14', 'DELIVERED', '2025-02-16', NULL),
(2, '2025-02-20', 'SHIPPED', '2025-02-21', 5.00),
(3, '2025-03-01', 'CANCELLED', NULL, NULL),
(4, '2025-03-15', 'PENDING', NULL, NULL),
(1, '2025-04-02', 'DELIVERED', '2025-04-04', NULL),
(5, '2025-04-18', 'DELIVERED', '2025-04-20', 15.00),
(2, '2025-05-09', 'PENDING', NULL, NULL);

INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(1, 1, 1, 89999.00), (1, 3, 2, 4999.00),
(2, 4, 3, 2499.00),
(3, 6, 1, 6999.00), (3, 7, 1, 3499.00),
(4, 2, 1, 74999.00),
(5, 3, 1, 4999.00),
(6, 1, 1, 89999.00),
(7, 6, 2, 6999.00), (7, 4, 1, 2499.00),
(8, 7, 1, 3499.00);

INSERT INTO payments (order_id, amount, method, paid_at, balance) VALUES
(1, 89996.10, 'UPI', '2025-01-10 11:02:00+05:30', 50000),
(2, 7497.00, 'CARD', '2025-02-14 19:31:00+05:30', 25000),
(3, 9972.60, 'NETBANKING', '2025-02-20 08:45:00+05:30', 10000),
(6, 89999.00, 'UPI', '2025-04-02 13:20:00+05:30', 8000),
(7, 14872.65, 'CARD', '2025-04-18 22:05:00+05:30', 3000);

INSERT INTO employees (full_name, department, salary, manager_id, hire_date) VALUES
('Nisha Rao', 'Engineering', 320000, NULL, '2019-04-01'), -- 1 CTO
('Karan Mehta', 'Engineering', 210000, 1, '2020-06-15'), -- 2
('Priya Nair', 'Engineering', 185000, 2, '2021-09-01'), -- 3
('Alok Gupta', 'Engineering', 185000, 2, '2022-01-10'), -- 4 same salary as 3
('Farah Ali', 'Sales', 150000, 1, '2021-03-22'), -- 5
('Vikram Singh', 'Sales', 120000, 5, '2023-07-05'), -- 6
('Tara Bose', 'Support', 90000, 5, '2024-02-19'); -- 7

```

Deliberate quirks in the data (they exist so the questions can bite):

- Sara Khan has a NULL city and no orders - powers NULL and anti-join questions.
- Orders 4, 5, 8 have no payment row - powers outer join and HAVING questions.
- discount_pct is NULL on 5 of 8 orders - powers COALESCE and NULL arithmetic.
- Employees 3 and 4 have identical salaries - powers RANK vs DENSE_RANK vs ROW_NUMBER.
- ProBook 15 has stock_qty = 0 - powers division-by-zero and NULLIF.
- categories is a three-level tree - powers recursive CTE.

Order totals (memorise these, many answers reference them):

order_id	customer	line total
1	Aarav	99997.00
2	Aarav	7497.00
3	Diya	10498.00
4	Rohan	74999.00
5	Meera	4999.00
6	Aarav	89999.00
7	John	16497.00
8	Diya	3499.00

Question 1

Difficulty: Easy

Category: SQL -> SELECT basics

Question

What does a SELECT statement do, and why is SELECT * discouraged in production code?

Answer

SELECT is a projection operator: it chooses which columns of which rows are returned. The engine reads rows from the source (table, index, or the output of a join), applies the filters, and then emits only the listed expressions.

SELECT * means "every column, in the table's physical order". It is fine when exploring data interactively. It is a liability in application code for five concrete reasons:

1. Breaks on schema change. Add a column and every SELECT * starts returning it. Code that maps by position (row[3]) silently reads the wrong field.
2. Wastes I/O and network. A TEXT blob you never use is still read from disk and shipped to the client.
3. Kills covering-index plans. If an index contains (customer_id, order_date) and you select only those two columns, Postgres can answer from the index alone (an *index-only scan). SELECT forces a heap fetch for every row.
4. Ambiguous in joins. Two tables both having unit_price produces two identically named output columns; most drivers then let you access only one of them.
5. Hides intent in review. The reader cannot tell which columns actually matter.

Example

You need a dropdown of customer names for an order form. You need two columns, not nine.

Code Example

```
-- Bad: ships every column including timestamps nobody renders
SELECT * FROM customers WHERE country = 'India';

-- Good: explicit projection, stable contract, index-friendly
SELECT customer_id,
       full_name
FROM   customers
WHERE  country = 'India'
ORDER BY full_name;
```

Output

```
customer_id | full_name
-----+-----
          1 | Aarav Sharma
          2 | Diya Patel
```

4 | Meera Iyer
3 | Rohan Verma
(4 rows)

Why Interviewers Ask This

It is the cheapest possible test of whether you have written code that survived a schema migration. A junior says "it returns all columns". A mid-level engineer mentions I/O and breakage. A senior mentions index-only scans and the API contract between database and application.

Common Mistakes

- Saying `SELECT *` is "slower" without saying why - vague answers read as memorised.
- Claiming it is always wrong. In an ad-hoc query, a `COUNT(*)`, or an `EXISTS` subquery it is perfectly correct.
- Forgetting that `SELECT *` inside a view freezes the column list at creation time in some engines, producing confusing drift later.

Best Practices

- Name every column in application queries; let code review see the contract.
- `SELECT COUNT(*)` is fine - no columns are actually materialised.
- Where a wide row is genuinely needed, define a view or a DTO so the column list lives in exactly one place.

Follow-up Questions

1. Is `SELECT COUNT(*)` slower than `SELECT COUNT(1)`? (No - the planner treats them alike.)
2. What is an index-only scan and how would you verify you got one?
3. How does `SELECT *` interact with `CREATE VIEW` and later `ALTER TABLE ADD COLUMN`?
4. Your ORM generates `SELECT *`. How would you fix that without rewriting every query?

Real-world Scenario

A team added an `internal_notes` TEXT column to customers for the support tool. The public customer API used `SELECT *` and serialised whatever it received. The next deploy leaked internal notes to customers through the JSON response. The fix was one line - an explicit column list - but the incident was a data-exposure postmortem.

Question 2

Difficulty: Medium

Category: SQL -> Logical query processing order

Question

In what order does SQL logically execute the clauses of a query? Why can you not use a column alias defined in SELECT inside the WHERE clause?

Answer

SQL is declarative, so written order is not execution order. The logical processing order is:

1. FROM pick the source tables
2. ON / JOIN match rows between them
3. WHERE filter individual rows <-- aliases from SELECT not visible yet
4. GROUP BY collapse rows into groups
5. HAVING filter groups <-- aggregates allowed here
6. SELECT compute output expressions <-- aliases are CREATED here
7. DISTINCT remove duplicate output rows
8. ORDER BY sort the result <-- aliases ARE visible
9. LIMIT/OFFSET slice the sorted result

A SELECT alias is created at step 6. WHERE runs at step 3, three steps earlier, so the name does not exist yet - hence column "x" does not exist. ORDER BY runs at step 8, after SELECT, which is why sorting by an alias works. (PostgreSQL and MySQL allow alias in ORDER BY; the standard also permits it there and nowhere earlier.)

The physical plan may differ wildly from this order - the optimiser reorders freely - but only in ways that cannot change the result.

Example

Computing a discounted order total once and filtering on it.

Code Example

```
-- FAILS: alias not yet defined when WHERE runs
-- SELECT order_id, quantity * unit_price AS line_total
-- FROM   order_items
-- WHERE  line_total > 50000;
-- ERROR:  column "line_total" does not exist

-- Fix 1: repeat the expression
SELECT order_id,
       quantity * unit_price AS line_total
FROM   order_items
WHERE  quantity * unit_price > 50000
ORDER BY line_total DESC;      -- alias legal here

-- Fix 2 (cleaner, no repetition): compute in a subquery / CTE, filter outside
SELECT order_id, line_total
FROM   (SELECT order_id, quantity * unit_price AS line_total
        FROM   order_items) t
WHERE  line_total > 50000
ORDER BY line_total DESC;
```


Output

```
order_id | line_total
-----+-----
      1 |    89999.00
      6 |    89999.00
      4 |    74999.00
(3 rows)
```

Why Interviewers Ask This

Almost every SQL bug a candidate will hit - alias errors, WHERE vs HAVING confusion, LIMIT picking the wrong rows, aggregates in the wrong clause - is explained by this one list. If you know the order, you can derive the rules instead of memorising them.

Common Mistakes

- Reciting the order but not being able to use it to explain the alias error.
- Believing the database physically executes in this order (it does not; the optimiser is free to reorder as long as results match).
- Putting a row filter in HAVING where WHERE belongs - it works but filters late and costs performance.

Best Practices

- Push filters as early as possible: row conditions in WHERE, group conditions in HAVING.
- Use a CTE when an expression is needed in both SELECT and WHERE - name it once.
- Remember LIMIT is last: LIMIT 10 without ORDER BY returns an arbitrary 10 rows.

Follow-up Questions

1. Where do window functions fit in this order? (After HAVING, alongside SELECT - which is why you cannot filter on a window function in WHERE.)
2. Why can you GROUP BY an ordinal position (GROUP BY 1) but not always by alias?
3. If the optimiser can reorder, why does the logical order matter at all?
4. Does LIMIT without ORDER BY ever return a stable result set?

Real-world Scenario

A reporting endpoint paginated with LIMIT 20 OFFSET 40 and no ORDER BY. It worked for months on a small table where Postgres happened to return rows in physical order. After a VACUUM FULL rewrote the heap, page 3 started showing rows that had already appeared on page 1. The bug was never in the pagination code - it was the missing final-step sort.

Question 3

Difficulty: Easy

Category: SQL -> WHERE vs HAVING

Question

What is the difference between WHERE and HAVING? Give a query that needs both.

Answer

Aspect	WHERE	HAVING
Operates on	individual rows	groups produced by GROUP BY
Runs at step	3 (before grouping)	5 (after grouping)
Aggregates allowed	no	yes
Can use SELECT alias	no	in some engines (MySQL yes, Postgres no)
Typical use	"only delivered orders"	"only customers with more than 2 orders"
Performance	filters early, less work downstream	filters late, after aggregation cost paid

Rule of thumb: **if the condition can be evaluated by looking at one row, it belongs in WHERE.** If it needs a COUNT, SUM, AVG, MIN or MAX across a group, it belongs in HAVING.

Both in one query reads naturally as: "Of the delivered orders only (WHERE), show me the customers who placed more than one (HAVING)."

Example

Find customers who have more than one delivered order, and their total spend.

Code Example

```
SELECT  c.customer_id,
        c.full_name,
        COUNT(*) AS delivered_orders,
        SUM(oi.quantity * oi.unit_price) AS total_spend
FROM    customers c
JOIN    orders o ON o.customer_id = c.customer_id
JOIN    order_items oi ON oi.order_id = o.order_id
WHERE   o.status = 'DELIVERED' -- row-level filter, applied first
GROUP BY c.customer_id, c.full_name
HAVING  COUNT(DISTINCT o.order_id) > 1 -- group-level filter, applied after
ORDER BY total_spend DESC;
```

Output

```
customer_id | full_name | delivered_orders | total_spend
-----+-----+-----+-----
1 | Aarav Sharma | 4 | 197493.00
(1 row)
```

Aarav has three delivered orders (1, 2, 6) spanning four line items, totalling $99997 + 7497 + 89999 = 197493$. John has only one delivered order so HAVING removes him.

Why Interviewers Ask This

It separates candidates who understand when each clause runs from those who pattern-match. The COUNT(*) vs COUNT(DISTINCT o.order_id) subtlety in the example - four line items but three orders - is a favourite trap.

Common Mistakes

- Putting WHERE status = 'DELIVERED' in HAVING. Postgres rejects it unless wrapped in an aggregate; MySQL historically allowed it and filtered late.
- Counting COUNT(*) after a join and reporting line items as if they were orders. Join fan-out inflates every aggregate.
- Forgetting that every non-aggregated SELECT column must appear in GROUP BY.

Best Practices

- Filter rows in WHERE so fewer rows enter the aggregation - measurable win on large tables.
- After any join, ask "did this multiply my rows?" and use COUNT(DISTINCT ...) or aggregate in a subquery before joining.
- Group by the primary key plus any functionally dependent columns; Postgres allows grouping by c.customer_id alone once it is the PK.

Follow-up Questions

1. Can HAVING be used without GROUP BY? (Yes - the whole result is one implicit group.)
2. Rewrite the query so the fan-out problem disappears entirely.
3. Which is faster: filtering in WHERE or in HAVING, and how would you prove it?
4. Why does COUNT(*) differ from COUNT(DISTINCT o.order_id) here?

Real-world Scenario

A "top customers" dashboard joined orders to order_items and used COUNT(*) as the order count. Customers who bought many items per order appeared to be the most frequent buyers, and the marketing team targeted the wrong segment for a loyalty campaign. The fix was COUNT(DISTINCT o.order_id).

Question 4

Difficulty: Easy

Category: SQL -> Aggregate functions

Question

Explain COUNT(*), COUNT(column), and COUNT(DISTINCT column). How do they treat NULL?

Answer

- COUNT(*) counts rows. It never inspects values, so NULLs are irrelevant.
- COUNT(col) counts non-NULL values of col. Every NULL is skipped.
- COUNT(DISTINCT col) counts distinct non-NULL values. NULL is still excluded, and duplicates collapse to one.

This is the general rule for all aggregates except COUNT(*): SUM, AVG, MIN, MAX ignore NULL inputs. The consequence that trips people up is AVG: AVG(col) is SUM(col) / COUNT(col), not SUM(col) / COUNT(*). Rows where col IS NULL do not dilute the average.

If every input is NULL, SUM and AVG return NULL (not 0), while COUNT returns 0.

Example

customers.city is NULL for Sara Khan, and orders.discount_pct is NULL for five of eight orders. Both expose the difference immediately.

Code Example

```
SELECT COUNT(*)           AS all_rows,           -- 6 customers
       COUNT(city)        AS with_city,          -- Sara's NULL skipped
       COUNT(DISTINCT country) AS distinct_countries,
       COUNT(last_support_contact) AS ever_contacted_support
FROM   customers;

-- The AVG trap, side by side
SELECT COUNT(*)           AS orders_total,
       COUNT(discount_pct) AS orders_with_discount,
       SUM(discount_pct)   AS discount_sum,
       ROUND(AVG(discount_pct), 2) AS avg_ignoring_nulls, -- /3
       ROUND(AVG(COALESCE(discount_pct,0)), 2) AS avg_treating_null_as_zero -- /8
FROM   orders;
```

Output

all_rows	with_city	distinct_countries	ever_contacted_support
6	5	3	2

orders_total	orders_with_discount	discount_sum	avg_ignoring_nulls	avg_treating_null_as_zero
8	3	30.00	10.00	3.75

30/3 = 10.00 versus 30/8 = 3.75. Same data, two very different business numbers.

Why Interviewers Ask This

It is a two-in-one probe: aggregate semantics and NULL semantics. The AVG divergence is

the single most common source of "the dashboard number is wrong" bugs, so interviewers use it as a proxy for data-correctness instincts.

Common Mistakes

- Believing COUNT(col) counts rows. It counts values.
- Assuming SUM of all-NULL returns 0. It returns NULL; wrap with COALESCE(SUM(x), 0) when the caller needs a number.
- Writing COUNT(DISTINCT a, b) and expecting portability - MySQL supports it, Postgres requires COUNT(DISTINCT (a, b)) on a row constructor.

Best Practices

- Prefer COUNT(*) for "how many rows"; it states intent and never surprises.
- Decide explicitly whether NULL means zero or means unknown, then encode that decision with COALESCE - do not let the default win by accident.
- Wrap outward-facing aggregates in COALESCE(..., 0) so APIs never return null totals.

Follow-up Questions

1. What does AVG return over an empty table, and how should an API respond?
2. How would you count customers without a city using only aggregates?
3. Does COUNT(1) differ from COUNT(*) in plan or performance?
4. How do NULLs behave in MIN/MAX versus in ORDER BY?

Real-world Scenario

A subscription analytics job reported average revenue per user using AVG(mrr). Trial users had mrr IS NULL rather than 0, so they were silently excluded and ARPU looked 40 percent healthier than reality. The board deck was wrong for a quarter. Fix: AVG(COALESCE(mrr, 0)) plus a NOT NULL DEFAULT 0 constraint on the column.

Question 5

Difficulty: Medium

Category: SQL -> NULL semantics

Question

Why does WHERE city = NULL return no rows? Explain SQL's three-valued logic.

Answer

NULL is not a value - it is a marker meaning "unknown". Comparing anything to an unknown yields UNKNOWN, not TRUE or FALSE. SQL therefore has three-valued logic: TRUE, FALSE, UNKNOWN.

WHERE keeps a row only when the predicate is exactly TRUE. city = NULL evaluates to UNKNOWN, so the row is discarded - which is why the query returns zero rows instead of "the rows with no city". The correct operators are IS NULL and IS NOT NULL.

Truth tables worth memorising:

A	B	A AND B	A OR B
TRUE	UNKNOWN	UNKNOWN	TRUE
FALSE	UNKNOWN	FALSE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Consequences that show up in real bugs:

- NULL = NULL is UNKNOWN, but NULL IS NOT DISTINCT FROM NULL is TRUE.
- NOT (city = 'Mumbai') does not include NULL cities.
- city NOT IN ('Mumbai', NULL) returns no rows ever - the NULL poisons the whole list.
- NULL in arithmetic propagates: 1000 * NULL is NULL.
- But UNIQUE constraints and GROUP BY treat NULLs as equal to each other, and ORDER BY sorts them together. The rules are deliberately inconsistent across features.

Example

Finding customers with no recorded city, and the NOT IN trap.

Code Example

```
-- Returns 0 rows: comparison with NULL is UNKNOWN
SELECT full_name FROM customers WHERE city = NULL;

-- Correct
SELECT full_name FROM customers WHERE city IS NULL;

-- The NOT IN + NULL trap: expected "everyone not in Mumbai", got nothing
SELECT full_name
FROM customers
WHERE city NOT IN ('Mumbai', NULL);      -- always empty

-- Safe alternatives
SELECT full_name
FROM customers
WHERE city IS DISTINCT FROM 'Mumbai';    -- NULL-safe inequality, keeps NULL rows

-- NULL-safe equality across engines
-- Postgres : a IS NOT DISTINCT FROM b
-- MySQL    : a <=> b
-- Portable : (a = b) OR (a IS NULL AND b IS NULL)
```

Output

```
-- WHERE city = NULL
full_name
-----
(0 rows)

-- WHERE city IS NULL
full_name
-----
Sara Khan
(1 row)
```

```
-- NOT IN ('Mumbai', NULL)
full_name
-----
(0 rows)

-- IS DISTINCT FROM 'Mumbai'
full_name
-----
Aarav Sharma
Rohan Verma
Meera Iyer
John Carter
Sara Khan
(5 rows)
```

Why Interviewers Ask This

NULL handling is the highest-density source of silent data corruption in SQL. A candidate who explains three-valued logic and then predicts the NOT IN behaviour is demonstrating that they debug from a model rather than by trial and error.

Common Mistakes

- Using = NULL or != NULL.
- Assuming NOT IN with a subquery is safe - if the subquery can yield a single NULL, the outer query returns nothing. This is the classic "my anti-join returns zero rows" bug.
- Thinking NULL and empty string "" are the same. They are not (except in Oracle).
- Believing NULL never counts as a duplicate - GROUP BY groups all NULLs together.

Best Practices

- Prefer NOT EXISTS over NOT IN for anti-joins - it is NULL-safe and usually plans better.
- Declare columns NOT NULL DEFAULT ... unless "unknown" is a genuine business state.
- Make the meaning of NULL explicit in the schema comment: unknown, not-applicable, or not-yet-known are three different things.
- In ORDER BY, state NULLS FIRST or NULLS LAST rather than relying on engine defaults.

Follow-up Questions

1. Rewrite city NOT IN (SELECT ...) as a NULL-safe anti-join.
2. Do UNIQUE constraints allow multiple NULLs? (Yes in Postgres/MySQL - they are not equal for uniqueness purposes; SQL Server allows only one.)
3. How does NULL sort in ORDER BY by default in Postgres versus MySQL?
4. What does COUNT(*) return for a group whose grouping key is NULL?

Real-world Scenario

A churn report used WHERE plan NOT IN (SELECT plan FROM active_plans) to find customers on retired plans. Someone inserted a row with plan = NULL into active_plans. Overnight the report returned zero churned customers, and the retention team assumed a great month. Nothing errored - the query was simply always UNKNOWN. Rewritten with NOT EXISTS, it worked again.

Question 6

Difficulty: Easy

Category: SQL -> COALESCE, NULLIF

Question

What do COALESCE and NULLIF do? How do you use them together to avoid division by zero?

Answer

COALESCE(a, b, c, ...) returns the first non-NULL argument, evaluated left to right, and short-circuits. It is the ANSI-standard way to supply a default. Engine-specific cousins: ISNULL (SQL Server, two args only), IFNULL (MySQL), NVL (Oracle). Prefer COALESCE - it is portable and takes any number of arguments.

NULLIF(a, b) returns NULL if a = b, otherwise returns a. It is the inverse tool: it creates a NULL deliberately.

The idiom $x / \text{NULLIF}(y, 0)$ is the standard division-by-zero guard: when y is 0 the denominator becomes NULL, so the whole expression is NULL instead of raising division by zero. Wrap it in COALESCE to present 0 to the caller:

```
COALESCE(x / NULLIF(y, 0), 0)
```

This is safer than a CASE WHEN y = 0 guard because it cannot be defeated by the optimiser evaluating branches in an unexpected order.

Example

Compute the effective total per order (discount is often NULL), and a stock turnover ratio where stock_qty can legitimately be 0 (ProBook 15).

Code Example

```
-- 1. COALESCE supplies a default for a NULL discount
SELECT o.order_id,
       o.discount_pct AS raw_discount,
       COALESCE(o.discount_pct, 0) AS effective_discount,
       SUM(oi.quantity * oi.unit_price) AS gross,
       ROUND(SUM(oi.quantity * oi.unit_price)
             * (1 - COALESCE(o.discount_pct, 0) / 100), 2) AS net_payable
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.discount_pct
ORDER BY o.order_id;

-- 2. NULLIF prevents division by zero for the zero-stock product
SELECT p.name,
       p.stock_qty,
       SUM(oi.quantity) AS units_sold,
       ROUND(SUM(oi.quantity)::numeric / NULLIF(p.stock_qty, 0), 2) AS turnover,
       COALESCE(ROUND(SUM(oi.quantity)::numeric
                     / NULLIF(p.stock_qty, 0), 2), 0) AS turnover_safe
```



```

FROM   products p
JOIN   order_items oi ON oi.product_id = p.product_id
GROUP BY p.name, p.stock_qty
ORDER BY p.name;

```

Output

order_id	raw_discount	effective_discount	gross	net_payable
1	10.00	10.00	99997.00	89997.30
2		0.00	7497.00	7497.00
3	5.00	5.00	10498.00	9973.10
4		0.00	74999.00	74999.00
5		0.00	4999.00	4999.00
6		0.00	89999.00	89999.00
7	15.00	15.00	16497.00	14022.45
8		0.00	3499.00	3499.00

(8 rows)

name	stock_qty	units_sold	turnover	turnover_safe
Air Fryer 4L	20	3	0.15	0.15
Mechanical Keyboard	40	3	0.08	0.08
ProBook 15	0	1		0.00
USB-C Hub	75	4	0.05	0.05
UltraBook 14	12	2	0.17	0.17
Blender Pro	18	2	0.11	0.11

(6 rows)

Without NULLIF, the ProBook row aborts the entire query with ERROR: division by zero.

Why Interviewers Ask This

Every reporting query in existence needs these two functions. The NULLIF division guard in particular is a small piece of professional vocabulary - candidates who know it have written real analytics SQL rather than only tutorial SQL.

Common Mistakes

- Assuming COALESCE converts types freely. All arguments must be type-compatible; COALESCE(discount_pct, 'none') fails on a numeric column.
- Using COALESCE(x, 0) on a column where NULL genuinely means "unknown" - this fabricates data and skews averages.
- Reaching for ISNULL/IFNULL in code meant to be portable.
- Writing NULLIF(y, 0) in the numerator by mistake, silently nulling the result.

Best Practices

- COALESCE at the presentation boundary; keep NULL in storage where it means unknown.
- Standardise on COALESCE(x / NULLIF(y, 0), 0) as the team's safe-division idiom.
- Cast before dividing integers: SUM(q)::numeric / ... avoids integer truncation in Postgres.

Follow-up Questions

1. How does COALESCE differ from CASE WHEN x IS NULL THEN y ELSE x END? (Semantically

identical; COALESCE evaluates x once and is clearer.)

2. Does COALESCE short-circuit? What if a later argument is an expensive subquery?
3. When is NULLIF useful outside division - for example on empty strings?
4. Why might COALESCE on an indexed column prevent index usage?

Real-world Scenario

A conversion-rate panel divided signups by visits. One day a traffic-source row arrived with zero visits and the whole ETL job failed at 3am, leaving the dashboard blank for a day. The one-character-class fix - NULLIF(visits, 0) plus COALESCE(..., 0) - made the pipeline degrade gracefully instead of dying.

Question 7

Difficulty: Easy

Category: SQL -> CASE expressions

Question

Explain the CASE expression. What is the difference between the simple and searched forms?

Answer

CASE is SQL's conditional expression - it returns a value, so it can appear anywhere a value can: SELECT, WHERE, ORDER BY, GROUP BY, even inside an aggregate. It is not a statement and has no side effects.

Searched form - each branch has a full predicate. Most flexible; use this by default.

```
CASE WHEN qty = 0 THEN 'out of stock'
      WHEN qty < 10 THEN 'low'
      ELSE 'healthy' END
```

Simple form - one expression compared for equality against constants. Shorter, but only tests =, and it cannot test NULL (because NULL = NULL is UNKNOWN).

```
CASE status WHEN 'PENDING' THEN 1 WHEN 'SHIPPED' THEN 2 ELSE 9 END
```

Key semantics:

- Branches are evaluated top to bottom and the first TRUE wins - order matters, and overlapping conditions are silently shadowed.
- Omitting ELSE implies ELSE NULL. This is the single most common CASE bug.
- All result branches must share a compatible type; the engine determines one result type.

Example

Bucket products by stock health, and impose a custom (non-alphabetical) status sort order.

Code Example

```
-- Searched CASE for bucketing + simple CASE for a custom sort key
SELECT p.name,
       p.stock_qty,
       CASE WHEN p.discontinued THEN 'discontinued'
            WHEN p.stock_qty = 0 THEN 'out of stock'
            WHEN p.stock_qty < 10 THEN 'low'
            ELSE 'healthy'
       END AS stock_status
FROM   products p
ORDER  BY CASE WHEN p.stock_qty = 0 THEN 0 ELSE 1 END, -- urgent rows first
        p.stock_qty;

-- CASE inside ORDER BY to enforce a workflow order, not alphabetical
SELECT order_id, status
FROM   orders
ORDER  BY CASE status
           WHEN 'PENDING' THEN 1
           WHEN 'SHIPPED' THEN 2
           WHEN 'DELIVERED' THEN 3
           WHEN 'CANCELLED' THEN 4
        END,
        order_id;
```

Output

name	stock_qty	stock_status
ProBook 15	0	out of stock
Wireless Mouse	5	discontinued
UltraBook 14	12	healthy
Blender Pro	18	healthy
Air Fryer 4L	20	healthy
Mechanical Keyboard	40	healthy
USB-C Hub	75	healthy

(7 rows)

order_id	status
5	PENDING
8	PENDING
3	SHIPPED
1	DELIVERED
2	DELIVERED
6	DELIVERED
4	CANCELLED

(7 rows)

Note Wireless Mouse reports discontinued, not low, because that branch is listed first. Branch order encodes business priority.

Why Interviewers Ask This

CASE is the building block for conditional aggregation, pivoting, and custom sorts - three things every reporting task needs. The missing-ELSE question also tests NULL awareness.

Common Mistakes

- Forgetting ELSE and being surprised by NULL rows downstream.
- Using the simple form to test for NULL: CASE city WHEN NULL THEN ... never matches. Use the searched form with WHEN city IS NULL.
- Writing overlapping conditions in the wrong order so a branch is unreachable.
- Mixing types across branches ('N/A' and a number) and hitting a cast error.

Best Practices

- Always write an explicit ELSE, even ELSE NULL, to document intent.
- Prefer the searched form; the extra characters buy IS NULL, ranges, and compound tests.
- For long stable mappings, move the logic to a lookup table and join - easier to change without a deploy.

Follow-up Questions

1. How do you pivot rows into columns using CASE inside SUM?
2. Does CASE short-circuit, and can you rely on that to avoid an expensive branch?
3. What is FILTER (WHERE ...) in Postgres and how does it compare to CASE in aggregates?
4. Can you use CASE in a GROUP BY, and what are the readability trade-offs?

Real-world Scenario

An order-management UI sorted statuses alphabetically, so CANCELLED appeared above PENDING and warehouse staff kept processing cancelled orders first. A CASE in ORDER BY mapping status to workflow position fixed it without a schema change.

Question 8

Difficulty: Medium

Category: SQL -> Conditional aggregation and pivoting

Question

How do you pivot rows into columns in standard SQL - for example, order counts per status as one row per customer?

Answer

The portable technique is conditional aggregation: put a CASE inside an aggregate, one aggregate per output column.

```
SUM(CASE WHEN status = 'DELIVERED' THEN 1 ELSE 0 END) AS delivered
```

SUM(CASE ... 1 ELSE 0) and COUNT(CASE ... THEN 1 END) both work. With COUNT you omit ELSE, relying on COUNT ignoring NULL. SUM with an explicit ELSE 0 is easier to read

and returns 0 rather than 0 for empty groups either way.

PostgreSQL adds the cleaner ANSI FILTER clause, which is faster to read and can be slightly faster to execute:

```
COUNT(*) FILTER (WHERE status = 'DELIVERED') AS delivered
```

Engine-specific pivot helpers exist (PIVOT in SQL Server/Oracle, crosstab() in the Postgres tablefunc extension) but they require a known, fixed column list, and so does conditional aggregation. True dynamic pivots require generating SQL in application code - this is the answer interviewers are listening for when they ask "what if the statuses are not known in advance?".

Example

One row per customer, one column per order status, plus lifetime spend.

Code Example

```
SELECT c.full_name,
       -- portable form
       SUM(CASE WHEN o.status = 'PENDING' THEN 1 ELSE 0 END) AS pending,
       SUM(CASE WHEN o.status = 'SHIPPED' THEN 1 ELSE 0 END) AS shipped,
       SUM(CASE WHEN o.status = 'DELIVERED' THEN 1 ELSE 0 END) AS delivered,
       SUM(CASE WHEN o.status = 'CANCELLED' THEN 1 ELSE 0 END) AS cancelled,
       -- Postgres-only, same result, clearer intent
       COUNT(*) FILTER (WHERE o.status = 'DELIVERED') AS delivered_filter,
       -- conditional SUM: only count revenue from non-cancelled orders
       COALESCE(SUM(CASE WHEN o.status <> 'CANCELLED'
                       THEN oi_totals.line_total END), 0) AS realised_revenue
FROM   customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
LEFT JOIN (SELECT order_id, SUM(quantity * unit_price) AS line_total
           FROM order_items GROUP BY order_id) oi_totals
       ON oi_totals.order_id = o.order_id
GROUP BY c.customer_id, c.full_name
ORDER BY realised_revenue DESC;
```

Note the pre-aggregated subquery: joining order_items directly would fan out rows and inflate every count.

Output

full_name	pending	shipped	delivered	cancelled	delivered_filter	realised_revenue
Aarav Sharma	0	0	3	0	3	197493.00
John Carter	0	0	1	0	1	16497.00
Diya Patel	1	1	0	0	0	13997.00
Meera Iyer	1	0	0	0	0	4999.00
Rohan Verma	0	0	0	1	0	0.00
Sara Khan	0	0	0	0	0	0.00

(6 rows)

Rohan's only order was cancelled, so his realised revenue is 0 despite a 74999 order. Sara has no orders at all and still appears - that is the LEFT JOIN doing its job.

Why Interviewers Ask This

Pivoting is the most requested "real work" SQL skill in analytics-adjacent roles, and the question doubles as a test of join fan-out awareness and LEFT JOIN correctness.

Common Mistakes

- Joining the detail table directly and inflating counts (the fan-out trap).
- Using COUNT(CASE WHEN ... THEN 1 ELSE 0 END) - COUNT counts the zeros too, so every group returns the total row count. Either use SUM(...1 ELSE 0) or drop the ELSE.
- Using an INNER JOIN and silently dropping customers with no orders.
- Hard-coding statuses and then being unable to answer the dynamic-pivot follow-up.

Best Practices

- Pre-aggregate to one row per join key before joining, as shown.
- Use FILTER (WHERE ...) on Postgres for readability; keep CASE where portability matters.
- Wrap outward-facing sums in COALESCE(..., 0).
- For genuinely dynamic columns, generate the SQL in the application or return long-format rows and pivot in the presentation layer.

Follow-up Questions

1. How would you unpivot - turn four status columns back into rows? (UNION ALL, or LATERAL/CROSS APPLY.)
2. Why is COUNT(CASE ... ELSE 0 END) wrong?
3. How would you handle a status that does not exist yet without editing the query?
4. What breaks if you replace the LEFT JOINS with INNER JOINS here?

Real-world Scenario

A weekly ops report needed orders-by-status per warehouse. The first version joined order_items and reported item counts as order counts; warehouse managers were rated on inflated throughput numbers for two months. Pre-aggregating before the join corrected the figures, and the team added a regression test asserting the report's row count matched SELECT COUNT(*) FROM orders.

Question 9

Difficulty: Easy

Category: SQL -> Joins

Question

Explain INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN and CROSS JOIN. Draw the row-matching behaviour.

Answer

A join combines rows from two inputs based on a predicate. The join type decides what happens to rows that find no match.

A	B	INNER	LEFT	FULL
+-----+	+-----+	+-----+	+-----+	+-----+
a1	b1	a1 b1	a1 b1	a1 b1
a2	b2	a2 b2	a2 b2	a2 b2
a3	b4		a3 --	a3 --
+-----+	+-----+		+-----+	+-----+
				-- b4
				+-----+

Join	Keeps	Unmatched left	Unmatched right
INNER	only matching pairs	dropped	dropped
LEFT [OUTER]	all left rows	kept, right columns NULL	dropped
RIGHT [OUTER]	all right rows	dropped	kept, left columns NULL
FULL [OUTER]	everything	kept with NULLs	kept with NULLs
CROSS	Cartesian product	n/a - no predicate	n/a

RIGHT JOIN is LEFT JOIN with the tables swapped; most teams ban it in code review because reading a query with mixed directions is error-prone. CROSS JOIN is legitimate for generating combinations (calendar x product) and is a bug everywhere else - usually a forgotten ON.

MySQL has no FULL OUTER JOIN; emulate it with `LEFT ... UNION ALL ... RIGHT ... WHERE left key IS NULL`.

Example

Which customers have orders, and which have none? Sara Khan (customer 6) has zero orders, so she is the row that distinguishes the join types.

Code Example

```
-- INNER: only customers who actually ordered -> Sara disappears
SELECT c.full_name, COUNT(o.order_id) AS orders
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.full_name
ORDER BY c.full_name;

-- LEFT: every customer, zero for those without orders -> Sara appears with 0
SELECT c.full_name, COUNT(o.order_id) AS orders
FROM   customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.full_name
ORDER BY orders DESC, c.full_name;

-- CROSS JOIN with intent: every product x every status, for a coverage matrix
SELECT p.name, s.status
FROM   products p
CROSS JOIN (VALUES ('PENDING'),('SHIPPED')) AS s(status)
WHERE  p.product_id <= 2
ORDER BY p.name, s.status;
```

Output

```
-- INNER (5 rows, Sara missing)
```

full_name	orders
Aarav Sharma	3
Diya Patel	2
John Carter	1
Meera Iyer	1
Rohan Verma	1

```
-- LEFT (6 rows, Sara present with 0)
```

full_name	orders
Aarav Sharma	3
Diya Patel	2
John Carter	1
Meera Iyer	1
Rohan Verma	1
Sara Khan	0

```
-- CROSS
```

name	status
ProBook 15	PENDING
ProBook 15	SHIPPED
UltraBook 14	PENDING
UltraBook 14	SHIPPED

(4 rows)

COUNT(o.order_id) returns 0 for Sara, not 1 - because the outer-joined column is NULL and COUNT(column) skips NULLs. COUNT(*) would wrongly report 1.

Why Interviewers Ask This

Joins are the core of relational thinking, and the COUNT(*) vs COUNT(col) detail on an outer join separates people who have reasoned about it from people who have memorised syntax.

Common Mistakes

- Using COUNT(*) with a LEFT JOIN and reporting 1 instead of 0 for empty groups.
- Putting the right table's filter in WHERE instead of ON - this silently converts a LEFT JOIN into an INNER JOIN (see the next question).
- Assuming FULL OUTER JOIN exists everywhere.
- Accidental CROSS JOIN from a missing ON, producing millions of rows.

Best Practices

- Always qualify columns with table aliases; unqualified columns in joins are review blockers.
- Prefer explicit JOIN ... ON over comma-separated FROM a, b WHERE.
- Standardise on LEFT JOIN and reorder tables rather than mixing in RIGHT JOIN.

Follow-up Questions

1. Why does COUNT(o.order_id) give 0 but COUNT(*) gives 1 for Sara?
2. When is a CROSS JOIN the correct tool?

3. How do you emulate FULL OUTER JOIN on MySQL?
4. What is a semi-join and an anti-join, and which SQL constructs express them?

Real-world Scenario

A monthly active users report used INNER JOIN from users to events. Users who signed up but did nothing were invisible, so the activation funnel showed a 100 percent activation rate. One word - LEFT - restored several thousand zero-activity users and revealed a real onboarding problem.

Question 10

Difficulty: Medium

Category: SQL -> Joins, ON vs WHERE

Question

What is the difference between putting a condition in the ON clause versus the WHERE clause of a LEFT JOIN?

Answer

For an INNER JOIN the two are equivalent - the optimiser will even rewrite one into the other. For an outer join they mean completely different things:

- ON is evaluated while matching rows. A non-matching right row simply becomes NULL, and the left row survives.

- WHERE is evaluated after the join has produced its rows. The NULL-extended row is then tested, fails the predicate, and the left row disappears.

Net effect: ****a filter on the right table's column in WHERE silently turns your LEFT JOIN into an INNER JOIN.**** This is one of the most common SQL bugs in production reporting.

The exception is WHERE right_col IS NULL, which is the deliberate anti-join idiom: keep only left rows that found no match.

Example

"Every customer with their DELIVERED order count" - correct with ON, broken with WHERE.

Code Example

```
-- CORRECT: condition in ON. All 6 customers survive.
SELECT c.full_name,
       COUNT(o.order_id) AS delivered_orders
FROM   customers c
LEFT   JOIN orders o
      ON o.customer_id = c.customer_id
      AND o.status      = 'DELIVERED'      -- part of the match
GROUP BY c.full_name
```

```

ORDER BY delivered_orders DESC, c.full_name;

-- BROKEN: condition in WHERE. Degrades to INNER JOIN, only 2 customers survive.
SELECT c.full_name,
       COUNT(o.order_id) AS delivered_orders
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
WHERE  o.status = 'DELIVERED' -- kills NULL-extended rows
GROUP BY c.full_name
ORDER BY delivered_orders DESC, c.full_name;

-- DELIBERATE anti-join: customers who never ordered
SELECT c.full_name
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
WHERE  o.order_id IS NULL;

```

Output

```

-- CORRECT (6 rows)
full_name | delivered_orders
-----+-----
Aarav Sharma | 3
John Carter | 1
Diya Patel | 0
Meera Iyer | 0
Rohan Verma | 0
Sara Khan | 0

-- BROKEN (2 rows) - four customers vanished
full_name | delivered_orders
-----+-----
Aarav Sharma | 3
John Carter | 1

-- ANTI-JOIN
full_name
-----
Sara Khan
(1 row)

```

Why Interviewers Ask This

It is the single highest-yield SQL correctness question. It cannot be answered by pattern matching - you must actually understand that WHERE runs after the join. Interviewers often present the broken query and ask "why are rows missing?"

Common Mistakes

- Believing ON and WHERE are interchangeable because they are for INNER JOIN.
- Adding a date filter on the outer table in WHERE and then wondering why the report lost customers.
- Writing WHERE o.status = 'DELIVERED' OR o.status IS NULL as a workaround - it works but obscures intent; put the condition in ON.

Best Practices

- Rule: **join conditions and filters on the outer table go in ON; filters on the preserved

table go in WHERE.**

- When a LEFT JOIN result row count equals the INNER JOIN count, treat it as a smell and check for this bug.
- Prefer NOT EXISTS over the IS NULL anti-join pattern when readability matters; both are correct and usually plan identically.

Follow-up Questions

1. Why is WHERE o.order_id IS NULL the exception, and what is it called?
2. Are ON and WHERE interchangeable for INNER JOIN? Prove it with a plan.
3. How does this behave for FULL OUTER JOIN - which side is preserved?
4. Rewrite the correct query using a correlated subquery instead of a join.

Real-world Scenario

A subscription revenue report added WHERE invoices.status = 'PAID' to a LEFT JOIN from customers. Customers with no invoices at all silently dropped out, so churn looked flat while signups were quietly failing to convert. The row count fell from 12,400 to 9,100 and nobody noticed for six weeks because both numbers looked plausible.

Question 11

Difficulty: Medium

Category: SQL -> Self join

Question

What is a self join? Write a query listing each employee with their manager's name.

Answer

A self join is a join of a table to itself, using two different aliases so the engine treats them as two independent inputs. It is the standard way to relate rows within one table: employee to manager, product to its substitute, event to the previous event.

Requirements:

1. Two distinct aliases (e and m) - without them column references are ambiguous.
2. A LEFT JOIN if the relationship is optional. The CTO has manager_id IS NULL, and an INNER JOIN would silently drop the top of the hierarchy.

A self join reaches exactly one level. For arbitrary depth - the whole reporting chain - you need a recursive CTE (covered later in this section).

Example

employees.manager_id points at employees.employee_id. Nisha Rao is the CTO with no manager.

Code Example

```
-- One level of hierarchy. LEFT JOIN so the CTO is not lost.
SELECT e.employee_id,
       e.full_name           AS employee,
       e.department,
       COALESCE(m.full_name, '(none)') AS manager,
       -- flag employees paid more than their manager
       CASE WHEN e.salary > m.salary THEN 'YES' ELSE 'no' END AS out_earns_manager
FROM   employees e
LEFT   JOIN employees m ON m.employee_id = e.manager_id
ORDER  BY e.employee_id;

-- Self join to find pairs of colleagues in the same department on equal salary.
-- e1.employee_id < e2.employee_id avoids self-pairs and mirrored duplicates.
SELECT e1.full_name AS employee_a,
       e2.full_name AS employee_b,
       e1.department,
       e1.salary
FROM   employees e1
JOIN   employees e2
      ON e1.department = e2.department
      AND e1.salary    = e2.salary
      AND e1.employee_id < e2.employee_id;
```

Output

employee_id	employee	department	manager	out_earns_manager
1	Nisha Rao	Engineering	(none)	no
2	Karan Mehta	Engineering	Nisha Rao	no
3	Priya Nair	Engineering	Karan Mehta	no
4	Alok Gupta	Engineering	Karan Mehta	no
5	Farah Ali	Sales	Nisha Rao	no
6	Vikram Singh	Sales	Farah Ali	no
7	Tara Bose	Support	Farah Ali	no

(7 rows)

employee_a	employee_b	department	salary
Priya Nair	Alok Gupta	Engineering	185000.00

(1 row)

For Nisha, m.salary is NULL, so e.salary > NULL is UNKNOWN, the CASE falls to ELSE, and she correctly reads no rather than erroring.

Why Interviewers Ask This

Self joins test whether you see a table as a set of rows rather than as a fixed object. The e1.id < e2.id trick for de-duplicating pairs is a small, recognisable piece of craft.

Common Mistakes

- Using INNER JOIN and losing the root of the hierarchy.
- Omitting e1.id < e2.id, producing each pair twice plus every row matched to itself.
- Forgetting aliases entirely - ERROR: table name "employees" specified more than once.
- Assuming a self join can traverse multiple levels.

Best Practices

- Alias meaningfully: e/m for employee/manager beats a/b.
- Index the foreign key (manager_id) - self joins hammer it.
- Use < rather than <> for unordered pair generation.
- Escalate to a recursive CTE the moment "all levels" is required.

Follow-up Questions

1. How would you list the full management chain up to the CTO?
2. How do you find employees with no direct reports?
3. What index would you add to make this join efficient at a million rows?
4. How does a self join differ from a window function for "compare to previous row" problems?

Real-world Scenario

An HR system needed an approval-chain email at each level. The first implementation self-joined five times to support five levels, then broke when the org added a sixth. Replacing the fixed chain of self joins with a recursive CTE made depth unbounded and removed 60 lines of SQL.

Question 12

Difficulty: Medium

Category: SQL -> Window functions

Question

What is a window function? How does it differ from GROUP BY aggregation?

Answer

A window function computes a value across a set of rows related to the current row while preserving every input row. GROUP BY collapses rows; a window function annotates them.

GROUP BY	WINDOW FUNCTION
-----	-----
6 rows in	6 rows in
2 rows out (one per group)	6 rows out (each + group value)
detail is lost	detail is kept

Syntax:

```
function() OVER (  
  PARTITION BY expr    -- optional: divides rows into independent windows  
  ORDER BY expr        -- optional: orders rows within the window  
  ROWS/RANGE frame     -- optional: which rows within the partition count  
)
```

Window functions run after WHERE, GROUP BY and HAVING, and before ORDER BY.

That ordering has one critical consequence: ****you cannot filter on a window function in WHERE.**** Wrap the query in a subquery or CTE and filter outside. This is the number one window-function error message candidates hit.

Three families:

Family	Examples	Needs ORDER BY
Ranking	ROW_NUMBER, RANK, DENSE_RANK, NTILE	yes
Aggregate-as-window	SUM, AVG, COUNT, MAX OVER	optional
Offset / navigation	LAG, LEAD, FIRST_VALUE, LAST_VALUE	yes

Example

Show each employee's salary next to their department average and their share of the department payroll - impossible with GROUP BY alone without a self join.

Code Example

```
SELECT full_name,
       department,
       salary,
       -- department aggregate, but every row is preserved
       ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS dept_avg,
       salary - ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS diff_from_avg,
       SUM(salary) OVER (PARTITION BY department) AS dept_payroll,
       ROUND(100.0 * salary
             / SUM(salary) OVER (PARTITION BY department), 1) AS pct_of_dept,
       COUNT(*) OVER (PARTITION BY department) AS dept_headcount,
       -- running total across the whole company, ordered by hire date
       SUM(salary) OVER (ORDER BY hire_date
                        ROWS BETWEEN UNBOUNDED PRECEDING
                        AND CURRENT ROW) AS cumulative_payroll
FROM   employees
ORDER BY department, salary DESC;
```

Output

full_name	department	salary	dept_avg	diff_from_avg	dept_payroll	pct_of_dept	dept_headcount	cumulative_payroll
Nisha Rao	Engineering	320000.00	225000.00	95000.00	900000.00	35.6	4	320000.00
Karan Mehta	Engineering	210000.00	225000.00	-15000.00	900000.00	23.3	4	530000.00
Priya Nair	Engineering	185000.00	225000.00	-40000.00	900000.00	20.6	4	865000.00
Alok Gupta	Engineering	185000.00	225000.00	-40000.00	900000.00	20.6	4	1070000.00
Farah Ali	Sales	150000.00	135000.00	15000.00	270000.00	55.6	2	680000.00
Vikram Singh	Sales	120000.00	135000.00	-15000.00	270000.00	44.4	2	1190000.00
Tara Bose	Support	90000.00	90000.00	0.00	90000.00	100.0	1	1280000.00

(7 rows)

Why Interviewers Ask This

Window functions are the dividing line between basic and professional SQL. Interviewers use them to check whether you can express "compare a row to its group" without a self join, and whether you know the WHERE-cannot-see-windows rule.

Common Mistakes

- Trying to filter on a window alias in WHERE -> `ERROR: window functions are not allowed in WHERE`. Wrap in a subquery or CTE.
- Confusing PARTITION BY with GROUP BY. PARTITION BY does not reduce row count.
- Adding ORDER BY inside OVER for an aggregate without realising it creates a running total (default frame is UNBOUNDED PRECEDING to CURRENT ROW), not a partition total.
- Assuming LAST_VALUE gives the partition's last row - with the default frame it returns the current row. You need ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

Best Practices

- Name the window once with a WINDOW clause when reusing it:
WINDOW w AS (PARTITION BY department) then AVG(salary) OVER w.
- Always state the frame explicitly for running calculations - do not rely on defaults.
- Index the PARTITION BY and ORDER BY columns to let the planner skip a sort.

Follow-up Questions

1. Why can you not use a window function in WHERE, and what is the workaround?
2. What is the default window frame, and when does it bite you?
3. Difference between ROWS and RANGE framing with duplicate ORDER BY values?
4. Rewrite dept_avg using a correlated subquery and compare the plans.

Real-world Scenario

A payroll fairness audit needed each employee's salary versus their band average, for 40,000 employees. The original implementation ran one correlated subquery per row - 40,000 aggregate scans, about nine minutes. A single AVG(...) OVER (PARTITION BY band) computed all of it in one pass in under two seconds.

Question 13

Difficulty: Medium

Category: SQL -> ROW_NUMBER, RANK, DENSE_RANK

Question

Explain the difference between ROW_NUMBER(), RANK() and DENSE_RANK().

Answer

All three number rows within a window; they differ only in how they treat ties.

Function	Ties get	Gaps after ties	Values for salaries 320k, 210k, ~
ROW_NUMBER()	different numbers (arbitrary amo~	no	1, 2, 3, 4
RANK()	the same number	yes - skips	1, 2, 3, 3 -> next would be 5
DENSE_RANK()	the same number	no	1, 2, 3, 3 -> next would be 4

Mnemonics:

- ROW_NUMBER = "give me exactly one row per position" -> deduplication, pagination.
- RANK = "Olympic medals" -> two silvers means no bronze.
- DENSE_RANK = "distinct salary levels" -> ranks values, not rows.

Critical detail: ROW_NUMBER() is non-deterministic across ties. If Priya and Alok both earn 185000 and you order only by salary, which gets 3 and which gets 4 can change between runs. Always add a unique tiebreaker (ORDER BY salary DESC, employee_id) when the result feeds anything reproducible.

Example

Employees 3 and 4 have identical salaries by design - that is what makes the three functions diverge visibly.

Code Example

```
SELECT full_name,
       department,
       salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC, employee_id) AS row_num,
       RANK() OVER (ORDER BY salary DESC) AS rank_val,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_val,
       -- per-department ranking, the classic "top N per group" building block
       RANK() OVER (PARTITION BY department
                   ORDER BY salary DESC) AS dept_rank
FROM   employees
ORDER BY salary DESC, employee_id;
```

Output

full_name	department	salary	row_num	rank_val	dense_rank_val	dept_rank
Nisha Rao	Engineering	320000.00	1	1	1	1
Karan Mehta	Engineering	210000.00	2	2	2	2
Priya Nair	Engineering	185000.00	3	3	3	3
Alok Gupta	Engineering	185000.00	4	3	3	3
Farah Ali	Sales	150000.00	5	5	4	1
Vikram Singh	Sales	120000.00	6	6	5	2
Tara Bose	Support	90000.00	7	7	6	3

(7 rows)

Read the tie carefully: rank_val jumps from 3 to 5 (skipping 4), while dense_rank_val goes 3 then 4. That is the entire difference, and it is exactly what interviewers check.

Why Interviewers Ask This

It is the most common window-function question in existence because the answer is short, verifiable, and the tie behaviour cannot be bluffed. It also sets up the "top N per group" follow-up, which is a genuine daily task.

Common Mistakes

- Saying RANK and DENSE_RANK are "basically the same".
- Using ROW_NUMBER() for a leaderboard, so two people on identical scores get different positions arbitrarily.
- Using RANK() for pagination - gaps make WHERE rank BETWEEN 11 AND 20 return the wrong count of rows.
- Forgetting the unique tiebreaker, producing unstable output between runs.

Best Practices

- Pagination and de-duplication -> ROW_NUMBER() with a deterministic ORDER BY.
- Public leaderboards -> RANK() (fair) or DENSE_RANK() (compact), decided with the product owner and documented.
- Always add a unique column as the last ORDER BY term.

Follow-up Questions

1. Write "top 2 earners per department" using one of these. (Filter dept_rank <= 2 in an outer query.)
2. Which of the three would you use to delete duplicate rows, and how?
3. What does NTILE(4) do, and how does it split 7 rows?
4. How do you get "top N per group" without window functions? (Correlated subquery or LATERAL.)

Real-world Scenario

A gaming leaderboard used ROW_NUMBER() over score. Players tied on score saw their rank flip-flop between page refreshes because the plan's tie order was unstable, and support tickets accused the company of rigging ranks. Switching to DENSE_RANK() and adding first_achieved_at as a tiebreaker made ranks stable and explainable.

Question 14

Difficulty: Medium

Category: SQL -> LAG, LEAD

Question

What do LAG() and LEAD() do? Use them to compute month-over-month growth.

Answer

LAG(expr, offset, default) reads a value from a previous row in the window;
LEAD(expr, offset, default) reads from a following row. Both require ORDER BY inside
OVER - "previous" is meaningless without an order.

- offset defaults to 1.

- default is returned instead of NULL when the target row does not exist (first row for LAG, last for LEAD). Supplying it is what turns a NULL-riddled result into a clean one.

They replace the classic self-join-on-n-1 pattern, which is both slower and wrong whenever the sequence has gaps (missing months, deleted ids).

Period-over-period growth is the canonical use:

$$\text{growth \%} = (\text{current} - \text{previous}) / \text{previous} * 100$$

Guard the denominator with NULLIF(previous, 0).

Example

Monthly revenue from delivered and shipped orders, with the previous month's value, absolute delta and percentage growth.

Code Example

```
WITH monthly AS (  
  SELECT DATE_TRUNC('month', o.order_date)::date      AS month,  
         SUM(oi.quantity * oi.unit_price)             AS revenue  
  FROM   orders o  
  JOIN   order_items oi ON oi.order_id = o.order_id  
  WHERE  o.status <> 'CANCELLED'  
  GROUP BY DATE_TRUNC('month', o.order_date)  
)  
SELECT month,  
       revenue,  
       LAG(revenue) OVER (ORDER BY month)            AS prev_month,  
       LEAD(revenue) OVER (ORDER BY month)           AS next_month,  
       revenue - LAG(revenue) OVER (ORDER BY month)   AS delta,  
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))  
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS growth_pct,  
       -- default argument: report the first month's delta against 0 instead of NULL  
       revenue - LAG(revenue, 1, 0::numeric) OVER (ORDER BY month) AS delta_no_null  
FROM   monthly  
ORDER BY month;
```

Output

month	revenue	prev_month	next_month	delta	growth_pct	delta_no_null
2025-01-01	99997.00		17995.00			99997.00
2025-02-01	17995.00	99997.00	4999.00	-82002.00	-82.0	-82002.00

2025-03-01		4999.00		17995.00		106496.00		-12996.00		-72.2		-12996.00
2025-04-01		106496.00		4999.00		3499.00		101497.00		2030.3		101497.00
2025-05-01		3499.00		106496.00				-102997.00		-96.7		-102997.00

(5 rows)

February = order 2 (7497) + order 3 (10498) = 17995. April = order 6 (89999) + order 7 (16497) = 106496. March excludes cancelled order 4, leaving only order 5 at 4999.

Why Interviewers Ask This

Every analytics and finance team needs period comparisons, and the naive self-join solution breaks on gaps. LAG/LEAD fluency signals you have done real reporting work. The NULLIF guard and the default argument are the details that distinguish a polished answer.

Common Mistakes

- Omitting ORDER BY inside OVER -> the notion of "previous" is undefined and engines error or return arbitrary rows.
- Self-joining on month = month - 1 and losing rows whenever a month has no data.
- Dividing by the previous value without NULLIF, so one zero month kills the query.
- Expecting LAG to skip NULLs - it does not; it returns whatever is in the offset row.

Best Practices

- Pre-aggregate into a CTE first, then window over it - clearer and avoids fan-out.
- For sparse series, generate a complete calendar with generate_series and LEFT JOIN so missing months become explicit zeros before you apply LAG.
- Use the third default argument to eliminate leading NULLs at the source.

Follow-up Questions

1. How would you fill missing months so growth is not misleading?
2. Compute a 3-month moving average using a window frame.
3. What is the difference between LAG(x, 2) and LAG(LAG(x))? (The latter is not legal - window functions cannot nest.)
4. How would you find the longest streak of consecutive growing months?

Real-world Scenario

A retention dashboard compared each cohort's month N to month N-1 via a self join on month_index - 1. Cohorts with a zero-activity month had that month missing entirely, so the join silently compared month 3 to month 1 and reported impossible retention above 100 percent. Rebuilt with a generated calendar plus LAG, the anomalies disappeared.

Question 15

Difficulty: Medium
Category: SQL -> CTE

Question

What is a CTE (WITH clause)? How does it compare to a subquery and to a temporary table?

Answer

A Common Table Expression is a named result set that exists for the duration of one statement:

```
WITH name AS ( SELECT ... )
SELECT ... FROM name;
```

Aspect	CTE	Inline subquery	Temp table
Lifetime	one statement	one statement	session / transaction
Reusable in same query	yes, by name	no - must repeat it	yes
Readability	high, top-to-bottom	drops off when nested	high
Indexable	no	no	yes
Statistics for planner	limited	limited	full, after ANALYZE
Recursion	yes (WITH RECURSIVE)	no	via loop

Materialisation is the detail that matters in interviews. Before PostgreSQL 12 every CTE was an optimisation fence - always materialised, no predicate push-down, which made CTEs a performance trap. From Postgres 12 a CTE referenced once is inlined by default, and you control it explicitly with MATERIALIZED / NOT MATERIALIZED. SQL Server and Oracle have always inlined CTEs (they are essentially macros there). MySQL supports CTEs from 8.0.

So the modern answer is: **CTEs are for readability and reuse; they are not a performance feature, and on Postgres you can now choose the materialisation behaviour.**

Example

A three-stage pipeline - order totals, then customer totals, then classification - written as readable steps instead of nested subqueries.

Code Example

```
WITH order_totals AS (          -- stage 1: one row per order
    SELECT o.order_id,
           o.customer_id,
           o.status,
           SUM(oi.quantity * oi.unit_price) AS order_value
    FROM   orders o
    JOIN   order_items oi ON oi.order_id = o.order_id
    GROUP BY o.order_id, o.customer_id, o.status
),
customer_totals AS (           -- stage 2: one row per customer, reuses stage 1
    SELECT c.customer_id,
           c.full_name,
           COUNT(ot.order_id) FILTER (WHERE ot.status <> 'CANCELLED') AS valid_orders,
           COALESCE(SUM(ot.order_value)
                    FILTER (WHERE ot.status <> 'CANCELLED'), 0) AS lifetime_value
    FROM   customers c
    LEFT JOIN order_totals ot ON ot.customer_id = c.customer_id
    GROUP BY c.customer_id, c.full_name
)
```

```

SELECT full_name,          -- stage 3: classify
       valid_orders,
       lifetime_value,
       CASE WHEN lifetime_value >= 100000 THEN 'VIP'
            WHEN lifetime_value >= 10000 THEN 'regular'
            WHEN lifetime_value > 0 THEN 'occasional'
            ELSE 'never purchased'
       END AS segment
FROM   customer_totals
ORDER  BY lifetime_value DESC;

-- Postgres 12+: force or prevent materialisation explicitly
-- WITH order_totals AS MATERIALIZED ( ... ) -- compute once, fence it
-- WITH order_totals AS NOT MATERIALIZED ( ... ) -- always inline

```

Output

full_name	valid_orders	lifetime_value	segment
Aarav Sharma	3	197493.00	VIP
John Carter	1	16497.00	regular
Diya Patel	2	13997.00	regular
Meera Iyer	1	4999.00	occasional
Rohan Verma	0	0.00	never purchased
Sara Khan	0	0.00	never purchased

(6 rows)

Rohan lands in "never purchased" because his single order was cancelled - the FILTER excluded it. Whether that is the right business rule is exactly the kind of clarifying question an interviewer wants you to ask.

Why Interviewers Ask This

CTEs are how maintainable analytical SQL is written. The materialisation-fence history is a sharp senior-level discriminator: many candidates still repeat "CTEs are slower in Postgres", which has been outdated since version 12.

Common Mistakes

- Believing a CTE is always materialised (or never) - the answer is version and engine specific.
- Chaining eight CTEs and assuming the planner will optimise across all of them.
- Using a CTE where an index was the real fix.
- Forgetting that a CTE cannot be indexed - for repeated heavy reuse a temp table may win.

Best Practices

- One CTE per logical step, named after what it produces, not t1/t2.
- Keep chains under roughly five stages; beyond that, a view or materialised view is clearer.
- On Postgres, benchmark MATERIALIZED versus NOT MATERIALIZED before assuming.
- Promote a CTE used by many queries to a view; promote an expensive one to a materialised view.

Follow-up Questions

1. When would a temp table beat a CTE?
2. What changed for CTEs in PostgreSQL 12?

3. Can a CTE be used in INSERT/UPDATE/DELETE, and what is RETURNING plus a CTE good for?
4. How does a CTE differ from a view in lifetime and permissions?

Real-world Scenario

A nightly billing job used a seven-stage CTE chain over 40 million invoice rows and took 50 minutes. Marking the two most reused stages MATERIALIZED - so they were computed once instead of re-inlined into three branches each - cut it to 11 minutes with no logic change.

Question 16

Difficulty: Hard

Category: SQL -> Recursive CTE

Question

Write a recursive CTE that returns the full category tree with each category's depth and path. Explain how recursion terminates.

Answer

WITH RECURSIVE has exactly two parts joined by UNION ALL:

```
WITH RECURSIVE t AS (  
    <anchor member>           -- runs once, seeds the result  
    UNION ALL  
    <recursive member>       -- references t, runs repeatedly  
)
```

Execution model:

1. Evaluate the anchor -> put rows in the working table.
2. Evaluate the recursive member using only the rows produced by the previous iteration.
3. Append those rows to the result and make them the new working table.
4. Repeat until an iteration produces zero rows. That is the termination condition.

So recursion stops naturally when no child rows match - not because of a counter. If the data contains a cycle (A is a parent of B, B is a parent of A) it never terminates. Two defences:

- A depth guard: WHERE depth < 10.
- A path array with WHERE NOT node = ANY(path) to detect revisits.

Note UNION ALL versus UNION: UNION de-duplicates each iteration, which masks cycles but is much slower. Use UNION ALL plus an explicit cycle guard.

Example

categories is a three-level tree: Electronics -> Computers -> Laptops / Accessories, and Home -> Kitchen.

Code Example

```
WITH RECURSIVE category_tree AS (  
  -- ANCHOR: the roots (no parent)  
  SELECT category_id,  
         name,  
         parent_id,  
         1 AS depth,  
         name::text AS path,  
         ARRAY[category_id] AS id_path  
  FROM categories  
  WHERE parent_id IS NULL  
  
  UNION ALL  
  
  -- RECURSIVE: children of whatever the previous iteration produced  
  SELECT c.category_id,  
         c.name,  
         c.parent_id,  
         ct.depth + 1,  
         ct.path || ' > ' || c.name,  
         ct.id_path || c.category_id  
  FROM categories c  
  JOIN category_tree ct ON c.parent_id = ct.category_id  
  WHERE ct.depth < 10 -- depth guard  
         AND NOT c.category_id = ANY(ct.id_path) -- cycle guard  
)  
SELECT category_id,  
       REPEAT(' ', depth - 1) || name AS indented_name,  
       depth,  
       path  
FROM category_tree  
ORDER BY path;  
  
-- Same technique upward: the management chain for Tara Bose (employee 7)  
WITH RECURSIVE chain AS (  
  SELECT employee_id, full_name, manager_id, 1 AS level  
  FROM employees WHERE employee_id = 7  
  UNION ALL  
  SELECT e.employee_id, e.full_name, e.manager_id, ch.level + 1  
  FROM employees e  
  JOIN chain ch ON e.employee_id = ch.manager_id  
)  
SELECT level, full_name FROM chain ORDER BY level;
```

Output

category_id	indented_name	depth	path
1	Electronics	1	Electronics
2	Computers	2	Electronics > Computers
4	Accessories	3	Electronics > Computers > Accessories
3	Laptops	3	Electronics > Computers > Laptops
5	Home	1	Home
6	Kitchen	2	Home > Kitchen

(6 rows)

level	full_name
1	Tara Bose
2	Farah Ali
3	Nisha Rao

(3 rows)

Iteration trace for the tree: anchor produces {Electronics, Home}; iteration 1 produces {Computers, Kitchen}; iteration 2 produces {Laptops, Accessories}; iteration 3 produces nothing -> stop.

Why Interviewers Ask This

Recursive CTEs are the standard test for hierarchical data - org charts, category trees, bill of materials, threaded comments, graph reachability. Explaining the termination rule and the cycle risk is what separates a memorised template from real understanding.

Common Mistakes

- Omitting RECURSIVE (required in Postgres; SQL Server does not use the keyword).
- Referencing the CTE twice in the recursive member - not allowed.
- No cycle guard, then an infinite loop that consumes disk in a production database.
- Using UNION instead of UNION ALL without knowing the de-duplication cost.
- Putting an aggregate or LEFT JOIN against the recursive term where the engine forbids it.

Best Practices

- Always include a depth guard, even when the data "cannot" have cycles.
- Carry an array path for both cycle detection and human-readable breadcrumbs.
- For very deep or hot trees, denormalise: materialised path (/1/2/4/), nested sets, or a closure table trades write cost for O(1) subtree reads.
- Index the parent column - the recursive join hits it once per iteration.

Follow-up Questions

1. How would you find all descendants of one node only?
2. Compare recursive CTE, adjacency list, materialised path and closure table.
3. How does CYCLE detection syntax work in Postgres 14+?
4. How would you compute the total stock of every product under a category subtree?

Real-world Scenario

An e-commerce catalogue allowed admins to re-parent categories. Someone made "Electronics" a child of "Laptops", creating a cycle. The nightly category rollup used a recursive CTE with no guard, ran until it filled the temp tablespace, and took the reporting replica offline. The postmortem added both a cycle guard in the query and a CHECK at write time.

Question 17

Difficulty: Medium

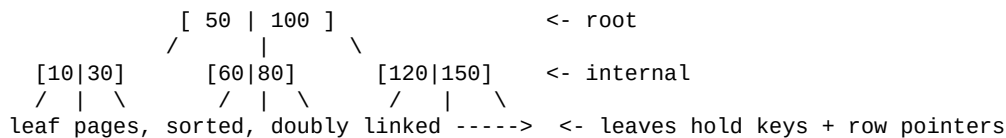
Category: SQL -> Indexes

Question

What is an index? Explain clustered versus non-clustered indexes and when an index hurts.

Answer

An index is an auxiliary data structure - almost always a B+ tree - that maps column values to row locations, turning an $O(n)$ table scan into an $O(\log n)$ lookup.



Clustered index - defines the physical order of the table's rows; the leaf level is the table. One per table. In SQL Server the PRIMARY KEY is clustered by default; in MySQL InnoDB it is the primary key, always. Range scans are extremely fast because rows are adjacent on disk.

Non-clustered (secondary) index - a separate structure holding the key plus a pointer to the row. Many per table. Requires a second step to fetch the row unless the index covers the query.

PostgreSQL is the important exception: it has no clustered indexes. All indexes are secondary, pointing at a heap; the CLUSTER command is a one-time physical reorder that is not maintained.

	Clustered	Non-clustered
Count per table	1	many
Stores	the rows themselves	key + row pointer
Extra lookup	none	yes, unless covering
Range queries	fastest	slower
Insert cost	can cause page splits	index maintenance only

When an index hurts:

- Every INSERT, UPDATE and DELETE must also update every affected index - write amplification. A table with 12 indexes writes 13 structures.
- Low-cardinality columns (a boolean, a two-value status) are usually not worth it; the planner will prefer a sequential scan anyway.
- Small tables: a scan of 200 rows beats index overhead.
- Unused indexes still cost writes, storage and backup time.
- Wrapping the column in a function (WHERE LOWER(email) = ...) makes a plain index unusable - you need an expression index.

Example

orders is queried by customer plus date constantly. A composite index serves both the filter and the sort.

Code Example

```
-- Composite index: column order matters. Leftmost-prefix rule.
```

```

CREATE INDEX idx_orders_customer_date
    ON orders (customer_id, order_date DESC);

-- USES the index (leading column present)
EXPLAIN ANALYZE
SELECT order_id, order_date, status
FROM   orders
WHERE  customer_id = 1
ORDER BY order_date DESC;

-- CANNOT use it efficiently: skips the leading column
SELECT order_id FROM orders WHERE order_date = '2025-04-02';

-- Partial index: only the rows anybody queries (open orders)
CREATE INDEX idx_orders_open
    ON orders (order_date)
    WHERE status IN ('PENDING', 'SHIPPED');

-- Covering index: answer entirely from the index, no heap fetch
CREATE INDEX idx_orders_cover
    ON orders (customer_id) INCLUDE (status, order_date);

-- Expression index: makes case-insensitive lookup indexable
CREATE INDEX idx_customers_email_lower ON customers (LOWER(email));

-- Find indexes nobody uses - candidates for deletion
SELECT relname AS table_name, indexrelname AS index_name, idx_scan AS times_used
FROM   pg_stat_user_indexes
WHERE  idx_scan = 0
ORDER BY relname;

```

Output

```

                        QUERY PLAN
-----
Index Scan using idx_orders_customer_date on orders
  Index Cond: (customer_id = 1)
  Planning Time: 0.180 ms
  Execution Time: 0.041 ms
(4 rows)

```

On a seven-row table Postgres may still choose a sequential scan - correctly, because it is cheaper. Demonstrating awareness of that is a plus, not a failure.

Why Interviewers Ask This

Indexing is where SQL knowledge meets production reality. Anyone can say "indexes make queries faster"; the signal is in the costs, the leftmost-prefix rule, and knowing that Postgres has no clustered index.

Common Mistakes

- "Add an index to every column." This is a write-performance disaster.
- Not knowing composite index column order matters - (a, b) serves WHERE a, and WHERE a AND b, but not WHERE b alone.
- Claiming Postgres primary keys are clustered.
- Forgetting that a function on the indexed column disables the index.
- Ignoring index bloat and never checking pg_stat_user_indexes.

Best Practices

- Index for your actual query patterns, verified with EXPLAIN ANALYZE and pg_stat_statements.
- Put the equality column first in a composite index, the range or sort column second.
- Use partial indexes for skewed data - the "open orders" case is often 1 percent of the table.
- Create indexes with CONCURRENTLY in production to avoid write locks.
- Audit and drop unused indexes on a schedule.

Follow-up Questions

1. Why does (customer_id, order_date) not help WHERE order_date = ??
2. What is a covering index and how do you confirm an index-only scan?
3. When would a hash, GIN or BRIN index beat a B-tree?
4. What is index bloat and how does REINDEX CONCURRENTLY help?
5. How do indexes interact with NULL values?

Real-world Scenario

A checkout service degraded after a growth push. pg_stat_user_indexes showed 14 indexes on orders, 9 of them never scanned - accumulated from years of one-off reports. Each insert was maintaining all 14. Dropping the unused ones cut p99 insert latency from 40ms to 9ms without touching a single query.

Question 18

Difficulty: Medium

Category: SQL -> Transactions, ACID

Question

Explain ACID. Show a transaction that must be atomic and how you would implement it safely.

Answer

ACID is the set of guarantees a relational database makes about a transaction - a unit of work that either fully happens or does not happen at all.

Property	Guarantee	Mechanism	Failure it prevents
Atomicity	all statements or none	undo log / rollback segment	money debited but never cre~
Consistency	constraints hold before an~	CHECK, FK, UNIQUE, triggers	negative stock, orphan rows
Isolation	concurrent transactions do~	locks, MVCC	lost updates, dirty reads
Durability	committed data survives a ~	write-ahead log flushed to~	"we committed it" then powe~

The classic example is a transfer: debit one account, credit another. A crash between the two statements without atomicity destroys money. The BEGIN ... COMMIT block makes both visible at the same instant, and ROLLBACK undoes everything if any step fails.

Durability specifically means the WAL is fsynced before COMMIT returns. This is why synchronous_commit = off is faster and why it can lose the last few transactions on power failure - a trade-off interviewers like to probe.

Example

Placing an order must do three things together: decrement stock, insert the order, record the payment. Any partial application corrupts the business state.

Code Example

```
-- Safe order placement. Either all three effects happen, or none do.
BEGIN;

-- 1. Reserve stock. The WHERE clause is the concurrency guard:
-- it only succeeds if stock is still available.
UPDATE products
SET    stock_qty = stock_qty - 1
WHERE  product_id = 1
      AND stock_qty >= 1;

-- If zero rows were updated, stock ran out -> abort.
-- In application code: if (rowsAffected == 0) throw -> ROLLBACK.

-- 2. Create the order
INSERT INTO orders (customer_id, order_date, status)
VALUES (2, CURRENT_DATE, 'PENDING')
RETURNING order_id;          -- say it returns 9

-- 3. Line item + payment, using that id
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (9, 1, 1, 89999.00);

INSERT INTO payments (order_id, amount, method)
VALUES (9, 89999.00, 'UPI');

COMMIT;      -- atomic: all four statements become visible together
-- ROLLBACK; -- on any error: nothing happened at all


// Node.js with node-postgres: the correct try/catch/finally shape
const client = await pool.connect();
try {
  await client.query('BEGIN');

  const stock = await client.query(
    `UPDATE products SET stock_qty = stock_qty - $2
     WHERE product_id = $1 AND stock_qty >= $2`,
    [productId, qty]
  );
  if (stock.rowCount === 0) throw new Error('OUT_OF_STOCK'); // triggers rollback

  const { rows } = await client.query(
    `INSERT INTO orders (customer_id, order_date, status)
     VALUES ($1, CURRENT_DATE, 'PENDING') RETURNING order_id`,
    [customerId]
  );
  const orderId = rows[0].order_id;

  await client.query(
    `INSERT INTO order_items (order_id, product_id, quantity, unit_price)
     VALUES ($1, $2, $3, $4)`,
    [orderId, productId, qty, price]
  );
}
```

```

    await client.query('COMMIT');
    return orderId;
} catch (err) {
    await client.query('ROLLBACK');    // atomicity in practice
    throw err;
} finally {
    client.release();                // never leak the connection
}

```

Output

```

-- successful path
UPDATE 1
  order_id
-----
      9
INSERT 0 1
INSERT 0 1
COMMIT

-- out-of-stock path
UPDATE 0
ROLLBACK
-- verification: nothing was written
SELECT COUNT(*) FROM orders WHERE order_id = 9; --> 0

```

Why Interviewers Ask This

ACID is the vocabulary of data correctness, and the interviewer wants to know whether you write BEGIN/COMMIT deliberately or rely on autocommit and hope. The WHERE stock_qty >= 1 guard is the detail that shows you have thought about two users buying the last item simultaneously.

Common Mistakes

- Reciting the acronym without being able to name a bug each property prevents.
- Reading stock, checking it in application code, then updating - a race condition. Do the check in the WHERE clause or lock the row with SELECT ... FOR UPDATE.
- No ROLLBACK in the error path, leaving the transaction open and holding locks.
- Not releasing the connection in finally - connection pool exhaustion under load.
- Long transactions spanning user think-time or external HTTP calls.

Best Practices

- Keep transactions short: no network calls, no user interaction inside them.
- Push invariants into the schema (CHECK (stock_qty >= 0)) so a logic bug cannot violate them.
- Make retries safe with an idempotency key so a client retry does not double-charge.
- Set an explicit statement_timeout and idle_in_transaction_session_timeout.

Follow-up Questions

1. What exactly does durability mean, and what does synchronous_commit = off give up?
2. How do two concurrent buyers of the last unit interleave, and who wins?
3. What is a savepoint and when is a nested rollback useful?
4. How do distributed transactions (2PC) or the Saga pattern change these guarantees?

5. Where does consistency in ACID differ from consistency in CAP?

Real-world Scenario

A wallet service credited the recipient in one transaction and debited the sender in another, for "simpler code". A deploy restarted pods between the two calls during a spike, and roughly 900 transfers credited without debiting - about 4 lakh rupees created out of nothing. The fix was a single transaction plus a reconciliation job that replays the ledger nightly.

Question 19

Difficulty: Hard

Category: SQL -> Isolation levels

Question

Name the four SQL isolation levels and the anomalies each prevents. Which does PostgreSQL use by default, and how does it differ from the standard?

Answer

Isolation controls what one transaction can see of another's uncommitted or concurrent work. Weaker isolation means more concurrency and more anomalies.

The anomalies:

- Dirty read - reading another transaction's uncommitted data.
- Non-repeatable read - re-reading the same row and getting a different value because someone else committed an UPDATE in between.
- Phantom read - re-running the same range query and finding new rows that another transaction inserted.
- Lost update - two transactions read, both write, the second overwrites the first.
- Write skew - both transactions read an overlapping set, each makes a decision that is individually valid, and the combination violates an invariant. Only SERIALIZABLE prevents it.

Level	Dirty read	Non-repeatable read	Phantom	Write skew
-----	-----	-----	-----	-----
READ UNCOMMITTED	possible	possible	possible	possible
READ COMMITTED	prevented	possible	possible	possible
REPEATABLE READ	prevented	prevented	possible (per standard)	possible
SERIALIZABLE	prevented	prevented	prevented	prevented

Engine reality, which is what interviews actually reward:

- PostgreSQL default is READ COMMITTED. Every statement sees a fresh snapshot. READ UNCOMMITTED is accepted but behaves as READ COMMITTED - Postgres cannot do dirty reads at all, because MVCC never exposes uncommitted tuples. Postgres REPEATABLE READ is snapshot isolation and does prevent phantoms, which is stricter than the standard requires. SERIALIZABLE uses Serializable Snapshot Isolation (SSI) and can abort with ERROR: could not serialize access - applications must retry.

- MySQL InnoDB default is REPEATABLE READ, and it blocks phantoms via next-key (gap) locks.
- SQL Server default is READ COMMITTED using shared locks, unless READ_COMMITTED_SNAPSHOT is enabled.

Example

Two support agents both refund the last available credit. Under READ COMMITTED each reads a balance of 100, each subtracts 100, and the account ends at -100. The invariant "balance >= 0" is violated even though each transaction was individually correct - write skew.

Code Example

```
-- ===== Demonstration of a non-repeatable read =====
-- Session A                               | Session B
BEGIN;                                     -- |
SELECT balance FROM payments              -- |
WHERE payment_id = 1;  --> 50000          -- |
                                           -- | BEGIN;
                                           -- | UPDATE payments SET balance = 40000
                                           -- | WHERE payment_id = 1;
                                           -- | COMMIT;

SELECT balance FROM payments              -- |
WHERE payment_id = 1;  --> 40000 (changed!) -- | READ COMMITTED: value changed
COMMIT;

-- Same script at REPEATABLE READ: the second SELECT still returns 50000,
-- because the whole transaction sees one snapshot taken at first statement.
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM payments WHERE payment_id = 1;  --> 50000
-- ... concurrent committed update happens ...
SELECT balance FROM payments WHERE payment_id = 1;  --> 50000 (stable)
COMMIT;

-- ===== Preventing lost update, two correct ways =====
-- (a) Pessimistic: lock the row until commit
BEGIN;
SELECT balance FROM payments WHERE payment_id = 1 FOR UPDATE; -- others block here
UPDATE payments SET balance = balance - 100 WHERE payment_id = 1;
COMMIT;

-- (b) Optimistic: make the write itself atomic and conditional
UPDATE payments
SET balance = balance - 100
WHERE payment_id = 1
AND balance >= 100; -- 0 rows affected => rejected, no negative balance

-- ===== SERIALIZABLE requires a retry loop =====
BEGIN ISOLATION LEVEL SERIALIZABLE;
-- ... reads and writes ...
COMMIT; -- may raise SQLSTATE 40001: could not serialize access
-- correct handling: catch 40001 and retry the whole transaction
```

Output

```
-- READ COMMITTED session A
balance
-----
50000
balance
-----
40000      <-- non-repeatable read
```

```
-- REPEATABLE READ session A
balance
-----
50000
balance
-----
50000      <-- stable snapshot

-- optimistic update when balance is only 50
UPDATE 0      <-- rejected safely, invariant preserved
```

Why Interviewers Ask This

This is the senior-level concurrency question. Anyone can list four level names; the signal is whether you can (1) name the anomaly each prevents, (2) know your engine's default and its deviations from the standard, and (3) describe the retry requirement of `SERIALIZABLE`.

Common Mistakes

- Saying Postgres defaults to `SERIALIZABLE`, or that `READ UNCOMMITTED` allows dirty reads in Postgres. Neither is true.
- Confusing non-repeatable read (a changed row) with phantom read (new rows).
- Choosing `SERIALIZABLE` without implementing retry on `SQLSTATE 40001` - the app then throws errors at users under load.
- Assuming `SELECT ... FOR UPDATE` inside `READ COMMITTED` prevents phantoms. It locks existing rows only; new inserts still appear.
- Believing higher isolation fixes bad schema design.

Best Practices

- Stay at the engine default; raise isolation only for the specific transaction that needs it.
- Prefer a conditional `UPDATE ... WHERE` (optimistic) or a version column over row locking - less blocking, no deadlock class.
- If you use `SERIALIZABLE`, wrap every transaction in a bounded retry with jitter.
- Enforce invariants with constraints as the final backstop: `CHECK (balance >= 0)`.
- Order writes consistently across the codebase to reduce deadlock probability.

Follow-up Questions

1. What is write skew and why does only `SERIALIZABLE` stop it?
2. How does MVCC let Postgres avoid dirty reads entirely?
3. Difference between `FOR UPDATE`, `FOR NO KEY UPDATE` and `FOR SHARE`?
4. What is a next-key lock in InnoDB and which anomaly does it target?
5. How would you implement optimistic locking with a version column?

Real-world Scenario

A seat-booking system checked availability with a `SELECT` and then inserted a booking, at `READ COMMITTED`. Under a flash sale two requests read "1 seat left" microseconds apart and both inserted - the flight was oversold by 30 seats across the event. The fix combined a unique constraint on `(flight_id, seat_no)` with an optimistic conditional update, so the database rejected the second writer instead of trusting application-side checks.

Question 20

Difficulty: Medium

Category: SQL -> Security, SQL injection

Question

What is SQL injection? Show a vulnerable query, exploit it, and fix it properly.

Answer

SQL injection happens when untrusted input is concatenated into SQL text, so the input can change the structure of the statement rather than only supplying a value. The database cannot tell the difference - by the time it parses the string, attacker text is indistinguishable from developer text.

The only complete fix is to stop building SQL from strings: use **parameterised queries / prepared statements**, where the driver sends the SQL template and the values on separate channels. The value is then never parsed as SQL, no matter what it contains.

Defence layers, strongest first:

1. Parameterised queries - always. This is the actual fix.
2. Allow-listing for identifiers. Table and column names cannot be parameterised, so if a sort column comes from the user, map it through a dictionary of permitted values.
3. Least privilege. The application role should not own the schema or hold SUPERUSER; injection then cannot DROP TABLE.
4. Input validation as defence in depth - never as the primary control.
5. Stored procedures help only if they themselves avoid dynamic SQL; EXECUTE inside a procedure is just as vulnerable.

Escaping quotes by hand is not a fix: encodings, numeric contexts, and second-order injection (stored data used to build a later query) all defeat it.

Example

A login form and a product search, both built by concatenation.

Code Example

```
// ===== VULNERABLE =====
// Never do this. String concatenation of user input.
async function loginUnsafe(email, password) {
  const sql = `SELECT customer_id, full_name FROM customers
              WHERE email = '${email}' AND password_hash = '${password}'`;
  return (await pool.query(sql)).rows;
}
// Attacker enters email: ' OR '1'='1' --
// Resulting SQL:
//   SELECT customer_id, full_name FROM customers
//   WHERE email = '' OR '1'='1' --' AND password_hash = '...'`
```

```
// The -- comments out the password check. Everyone logs in as the first row.

// Worse, with a stacked statement where the driver permits it:
// email = ''; DROP TABLE payments; --

// ===== SAFE: parameterised =====
async function loginSafe(email, passwordHash) {
  const sql = `SELECT customer_id, full_name
                FROM   customers
                WHERE  email = $1 AND password_hash = $2`;
  // $1/$2 are bound as VALUES; the string '' OR '1'='1' is just a string
  return (await pool.query(sql, [email, passwordHash])).rows;
}

// ===== SAFE: dynamic sort column via allow-list =====
const SORTABLE = {
  name: 'full_name',
  date: 'signup_date',
  city: 'city',
};
async function listCustomers(sortKey = 'name', dir = 'asc', limit = 20) {
  const column = SORTABLE[sortKey] ?? 'full_name'; // never user text
  const direction = dir.toLowerCase() === 'desc' ? 'DESC' : 'ASC';
  const sql = `SELECT customer_id, full_name, city
                FROM   customers
                ORDER  BY ${column} ${direction}
                LIMIT  $1`;
  return (await pool.query(sql, [limit])).rows;
}

-- Parameterised at the SQL level, for comparison
PREPARE find_customer (text) AS
  SELECT customer_id, full_name FROM customers WHERE email = $1;

EXECUTE find_customer ('aarav@example.com');
EXECUTE find_customer ('' OR '1'='1'); -- treated as a literal, returns 0 rows

-- Least privilege: the app role cannot drop anything
CREATE ROLE app_user LOGIN PASSWORD 'strong';
GRANT SELECT, INSERT, UPDATE ON orders, order_items TO app_user;
REVOKE ALL ON payments FROM app_user;
```

Output

```
-- vulnerable version with email = ' OR '1'='1' --
customer_id | full_name
-----+-----
1 | Aarav Sharma    <-- logged in with no password

-- parameterised version, same input
customer_id | full_name
-----+-----
(0 rows)          <-- treated as a literal string
```

Why Interviewers Ask This

It is the most exploited web vulnerability class in history and it is entirely preventable, so it is a direct test of professional hygiene. Interviewers also listen for whether you know the limits of parameterisation - identifiers cannot be bound - because that is where real applications still get breached.

Common Mistakes

- Believing an ORM makes you immune. Raw-SQL escape hatches (query, raw, whereRaw, @Query with string concatenation) reintroduce the hole.
- Escaping quotes manually or stripping keywords with a blocklist.
- Trying to parameterise a table or column name and then falling back to concatenation.
- Forgetting second-order injection: stored input reused later to build SQL.
- Running the app as the database owner, so a successful injection is catastrophic rather than merely bad.

Best Practices

- Parameterise everything; make string-concatenated SQL a lint error in CI.
- Allow-list identifiers for dynamic ORDER BY and dynamic table names.
- Grant the application role the minimum privileges; separate migration and runtime roles.
- Never store passwords with reversible or fast hashes - use Argon2id or bcrypt.
- Add a static analysis / SAST rule and a dependency scan; log and alert on query errors that smell like probing.

Follow-up Questions

1. Why can a table name not be a bind parameter?
2. Are stored procedures automatically safe? (No - dynamic EXECUTE inside is vulnerable.)
3. What is blind SQL injection and how is it detected without visible output?
4. How does least privilege limit blast radius after a successful injection?
5. What is second-order SQL injection?

Real-world Scenario

An internal admin tool let staff sort a report by clicking column headers, passing the column name straight into ORDER BY. A contractor's browser extension appended a payload to the query string, and the tool executed a subquery that exfiltrated the customer email table one row at a time via sort order. Parameterisation would not have helped - the fix was the allow-list map. The audit also found the tool was connecting as the schema owner.

SQL Coding Challenges (Set 1: Questions 1-20)

Attempt each in under 10 minutes, then compare. These are the shapes that recur constantly in screening rounds.

C1. Return the second-highest distinct salary from employees without using LIMIT/OFFSET.

```
SELECT MAX(salary) AS second_highest
FROM   employees
WHERE  salary < (SELECT MAX(salary) FROM employees);
-- 210000.00
```

C2. Find customers who have never placed an order, three different ways, and say which you

would ship.

```
-- (a) anti-join: fine, plans well
SELECT c.full_name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- (b) NOT EXISTS: preferred - NULL-safe and self-documenting
SELECT c.full_name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

-- (c) NOT IN: DANGEROUS - returns nothing if any customer_id is NULL
SELECT c.full_name FROM customers c
WHERE c.customer_id NOT IN (SELECT customer_id FROM orders);
-- Ship (b). Answer: Sara Khan
```

C3. Top 2 highest-paid employees per department.

```
SELECT full_name, department, salary
FROM (SELECT full_name, department, salary,
            DENSE_RANK() OVER (PARTITION BY department
                               ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk <= 2
ORDER BY department, salary DESC;
-- Engineering: Nisha 320000, Karan 210000
-- Sales:       Farah 150000, Vikram 120000
-- Support:     Tara 90000
```

C4. Delete duplicate rows keeping the lowest id (assume a table with repeats).

```
DELETE FROM some_table s
USING (SELECT id,
            ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
      FROM some_table) d
WHERE s.id = d.id AND d.rn > 1;
```

C5. Running total of revenue by order date.

```
SELECT o.order_date,
       SUM(oi.quantity * oi.unit_price) AS daily,
       SUM(SUM(oi.quantity * oi.unit_price)) OVER (ORDER BY o.order_date
                                                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM   orders o JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_date
ORDER BY o.order_date;
```

C6. Orders with no payment recorded (a reconciliation query every payments team runs).

```
SELECT o.order_id, o.status
FROM   orders o
WHERE  NOT EXISTS (SELECT 1 FROM payments p WHERE p.order_id = o.order_id)
       AND o.status <> 'CANCELLED';
-- orders 5 and 8 (both PENDING); order 4 excluded because it was cancelled
```

C7. For each customer, days between their first and most recent order.

```
SELECT c.full_name,
       MIN(o.order_date) AS first_order,
       MAX(o.order_date) AS last_order,
       MAX(o.order_date) - MIN(o.order_date) AS days_active
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.full_name
```

```

HAVING COUNT(*) > 1
ORDER BY days_active DESC;
-- Aarav: 2025-01-10 -> 2025-04-02 = 82 days
-- Diya: 2025-02-20 -> 2025-05-09 = 78 days

```

Revision Notes: SQL Questions 1-20

Clause execution order (the one list that explains most SQL errors)

FROM -> ON -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT

Mnemonic: Friendly Owls Will Gladly Help Small Ducks Order

Lunch. Aliases are born in SELECT, so only ORDER BY (and LIMIT) can see them.

NULL rules

Expression	Result
-----	-----
NULL = NULL	UNKNOWN (row dropped)
x NOT IN (1, NULL)	never returns rows
COUNT(col)	skips NULL
AVG(col)	divides by non-NULL count
SUM(all NULL)	NULL, not 0
GROUP BY on NULL	all NULLs form one group

Use IS NULL, IS DISTINCT FROM, NOT EXISTS, COALESCE, and x / NULLIF(y,0).

Join decision table

Need	Use
-----	-----
Only matching rows	INNER JOIN
All left rows, zeros for missing	LEFT JOIN + COUNT(right_col)
Rows with no match	NOT EXISTS (or LEFT JOIN ... IS NULL)
Filter the outer table	condition in ON, never WHERE
Row-within-table relationship	self join (one level) / recursive CTE (all levels)

The ON vs WHERE rule: a filter on the outer table in WHERE turns LEFT JOIN into INNER JOIN. Only WHERE right.key IS NULL is intentional.

Window function ties

320k, 210k, 185k, 185k	Result
-----	-----
ROW_NUMBER()	1, 2, 3, 4
RANK()	1, 2, 3, 3 (next = 5)
DENSE_RANK()	1, 2, 3, 3 (next = 4)

Windows run after HAVING, so never filter a window function in WHERE - wrap it.

Recursive CTE = anchor UNION ALL recursive member; stops when an iteration yields zero rows; always add a depth guard and a cycle guard.

Index quick rules - composite (a, b) serves a and a AND b, not b alone. Functions on the column disable the index. Every index taxes writes. Postgres has no clustered index.

ACID - Atomicity (all or none), Consistency (constraints), Isolation (concurrency), Durability (survives crash, via fsynced WAL).

Isolation defaults - Postgres READ COMMITTED (no dirty reads ever; REPEATABLE READ also blocks phantoms; SERIALIZABLE needs retry on SQLSTATE 40001). MySQL InnoDB REPEATABLE READ with gap locks.

Injection - parameterise values, allow-list identifiers, least privilege. An ORM is not a guarantee.

Common misconceptions cleared

1. "CTEs are always slower in Postgres" - false since version 12; single-reference CTEs inline.
 2. "SELECT COUNT(1) is faster than COUNT(*)" - identical plans.
 3. "Indexes only help" - they tax every write and can be ignored by the planner.
 4. "READ UNCOMMITTED gives dirty reads in Postgres" - it does not; MVCC forbids it.
 5. "HAVING and WHERE are interchangeable" - only accidentally, for non-aggregate predicates.
 6. "An ORM prevents SQL injection" - only until the first raw call.
-

BUILD STATUS AND ROADMAP

This book is being written in batches. This edition is Batch 1.

Section	Topic	Target	Written	Status
1	SQL	150	20	in progress
2	MERN Stack	120	0	pending
3	Internship and Fresher	100	0	pending
4	Computer Networks	100	0	pending
5	System Design	120	0	pending
6	HTML	60	0	pending
7	CSS	100	0	pending
8	JavaScript	150	0	pending
9	TypeScript	80	0	pending
10	C# and .NET	100	0	pending
11	Java	120	0	pending
12	Linux	100	0	pending
13	Data Structures and Algorithms	150	0	pending
14	Object-Oriented Programming	60	0	pending
15	DBMS	80	0	pending
16	Operating Systems	80	0	pending
17	Software Engineering Practice	50	0	pending
18	Behavioral and HR	100	0	pending
	Total	1720	20	1.2 percent

What Batch 1 contains

SQL questions 1 to 20, each with the full ten-part treatment, plus the shared sample database, seven coding challenges with solutions, and a revision sheet.

Coverage of the SQL syllabus so far:

- Covered: SELECT, projection cost, logical clause order, WHERE vs HAVING, aggregates and NULL, three-valued logic, COALESCE, NULLIF, CASE, conditional aggregation and pivoting, all five join types, ON vs WHERE on outer joins, self joins, window function fundamentals, ROW_NUMBER/RANK/DENSE_RANK, LAG/LEAD, CTEs and materialisation, recursive CTEs, indexes (clustered, non-clustered, composite, partial, covering, expression), ACID, isolation levels and concurrency anomalies, SQL injection.

- Still to come in Section 1: GROUP BY advanced (GROUPING SETS, ROLLUP, CUBE), LIMIT and keyset pagination, DISTINCT internals, set operators, all key types (primary, foreign, composite, candidate, super, unique), views and materialised views, stored procedures, functions, triggers, locks and deadlock resolution, normalisation 1NF to 5NF and denormalisation, EXPLAIN plan reading, query optimisation patterns, partitioning, sharding,

full-text search, JSON columns, and a long set of interview-style query problems.

How the remaining batches are produced

Each batch appends complete question blocks to interview-1000-questions.md and regenerates Interview_1000_Questions.pdf with generate_interview_pdf.py. Nothing already written is rewritten, so question numbers are stable and can be referenced across batches.

```
python3 generate_interview_pdf.py          # rebuild the PDF after any edit
```